

EduRev

Project List

Track J: React / Next.js · Node.js + Express / NestJS · MongoDB · REST + OpenAPI · JWT / OAuth · Docker · CI/CD **Track**

P: Django templates or React / Next.js · Python + Django · PostgreSQL · DRF + OpenAPI · Celery · Docker · CI/CD

Mapped courses: CSE326: INTERNET PROGRAMMING · INT252: WEB APP DEVELOPMENT WITH REACTJS · INT253: WEB DEVELOPMENT IN PYTHON USING DJANGO · INT257: MODERN WEB APPLICATION DEVELOPMENT

Section 1 – Common Engineering Standard: the stack, architecture, non-functional baseline, security, testing, documentation and Definition of Done for *both* tracks, plus the mapping from each requirement to the course that teaches it. Section 2 – scale bands and the weighted ranking of all twenty projects.

Section 3 – P01 to P20, each with all 22 sections. Sections 7, 8, 16, 18 and 19 are given separately for each track; every other section is stack-neutral and applies to both.

EduRev Project List · Dual Stack · P01–P20

Contents

1. Common Engineering Standard (CES)	1
1.1 Mandatory stack	1
1.2 Architecture rules	2
Non-functional baseline	2
1.4 Security baseline	2
1.5 Testing requirements	3
1.6 Documentation requirements	3
1.7 Definition of Done (every milestone)	4
Course mapping and permitted delivery paths	4
1.9 What no course teaches, and which track suits which project	5
1.10 Why Track P uses PostgreSQL and not MongoDB	5
2. Scale bands and weighted ranking	5
2.1 Scale bands and what they mean for a student team	5
2.2 Weighted ranking of all 20 projects	6
3. Project specifications – P01 to P20	7
P01 – Academic Risk Radar	8
P02 – Academic Advisor and Degree Planning System	12
P03 – Hostel Allocation and Roommate Matching	15
P04 – Hostel Room Exchange	18
P05 – Mess Demand Forecasting and Food-Waste Management	21
P06 – Placement Skill Matcher and Readiness Analyser	24
P07 – Health Services and Emergency Response	27
P08 – Research Intelligence and Collaboration Hub	30
P09 – Lost and Found Management System	33
P10 – Campus Issue Management and Network Quality Analysis	36
P11 – Alumni Mentorship and Career Network	39
P12 – CertiScholar: Document Verification and Scholarship Recommendation	42
P13 – Faculty Workload, Course Allocation and Timetable Engine	46
P14 – Course Registration and Elective Allocation Platform	50
P15 – Internship and AEDP Apprenticeship Management System	53
P16 – Capstone and Sponsored Project Management Platform	57
P17 – Student Earnings, Freelancing and Enterprise Tracking Platform (EdRev)	60
P18 – CO-PO Attainment and Accreditation Management System	64
P19 – AI-Assisted Question Bank and Assessment Platform	68
P20 – Campus Resource, Lab and Facility Booking Platform	71
Appendix – Track selection summary	74

1. Common Engineering Standard (CES)

Issue this section with every project brief. It is referenced by sections 7, 8, 15, 16 and 17 of every project specification in section 3, so those sections contain only the project-specific deltas.

A note on standards and student teams. These requirements are deliberately not softened for students. A team that ships without tests, without CI, without an API specification or without server-side authorisation has not learned the thing this programme exists to teach, and has produced software the institution cannot operate. What *is* adjusted for students is scope, not standard – fewer modules, built properly. Where a specific requirement is genuinely beyond a semester, the mentor records the deviation in week 2 and the panel assesses against the recorded scope.

A note on the two tracks. Every project in this document can be built on either track. The choice is made by the team in week 1 and recorded in the design document; it is not changed afterwards. The two tracks are not equivalent in difficulty for every project, and §1.9 says plainly which projects favour which track.

1.1 Mandatory stack

Language TypeScript on both client and server, no exceptions	Python 3.12 on the server; TypeScript or ES2022 JavaScript on the client
Framework Node.js 20+ with Express, or NestJS for projects marked Large or Enterprise	Django 5.x (LTS release), one Django app per functional module
Frontend React 18+ with Next.js (App Router); plain React + Vite only for internal admin tools	Django templates by default; React or Next.js against a DRF API where the project says so (§1.8)
UI Tailwind CSS + shadcn/ui or Material UI – one, consistently	Tailwind CSS, or a documented CSS architecture; a component library and design tokens before feature work
State/data TanStack Query for server state; Zustand or Context for client state	Server-rendered state by default; TanStack Query only on Paths P2 and P3
Database MongoDB 7+ with Mongoose or Prisma; replica set in production	PostgreSQL 16. Not MongoDB – see §1.10
Cache/queue Redis for cache, sessions and rate limiting; BullMQ for background jobs	Redis through Django's cache framework; Celery (or Django-Q on Small projects) for background jobs
Scheduled work BullMQ repeatable jobs – never an in-process cron	Celery Beat – never a cron calling manage.py inside the web container
API REST, documented with OpenAPI 3.1, versioned at /api/v1	Django REST Framework with drf-spectacular (OpenAPI 3), versioned at /api/v1. Required on Paths P2 and P3; optional on Path P1
Validation Zod on every endpoint Django Forms and DRF serializers on every endpoint; Pydantic where a service boundary needs it	
Auth JWT access + refresh, httpOnly cookies; OAuth2/OIDC for university SSO; TOTP MFA for admin roles	Django session auth on Path P1; SimpleJWT on Paths P2 and P3; django-allauth for SSO; TOTP MFA for admin roles
Authorisation RBAC with attribute checks, enforced server-side on every endpoint. UI hiding is not authorisation	Django groups and permissions <i>plus</i> an object-level check, enforced twice: as a permission and as a filtered get_queryset. UI hiding is not authorisation

Files S3-compatible object storage (S3 / R2 / MinIO), pre-signed URLs	Shared by both tracks: django-storages against the same, pre-signed URLs. Never MEDIA_ROOT on the app server
Search MongoDB Atlas Search or Meilisearch where the spec requires text search	PostgreSQL full-text search (SearchVector) and pg_trgm first; Meilisearch only where Postgres is genuinely not enough
Realtime Socket.IO or SSE where the spec requires live updates	Django Channels over ASGI where the spec requires live updates
Documents PDF via a server-side renderer; ExcelJS for spreadsheets	WeasyPrint for PDF (HTML and CSS in, PDF out); openpyxl for spreadsheets; python-docx for Word
Containers Docker, multi-stage builds; docker compose for local development – one command brings the stack up	Identical requirement; the compose file carries web, worker, PostgreSQL and Redis services
CI/CD GitHub Actions – lint, typecheck, test, build, deploy on merge. Required from M1	GitHub Actions – ruff, pytest-django, migration check, build, deploy on merge. Required from M1
Hosting Frontend on Vercel; backend on a container platform; MongoDB Atlas	Gunicorn behind Nginx (Uvicorn or Daphne where Channels is used), or Render / Railway; managed PostgreSQL
Observability Structured JSON logging (Pino), Sentry, uptime monitoring, /health and /ready	Structured logging via Python logging in JSON, Sentry, uptime monitoring, /health and /ready

1.2 Architecture rules

- Modular monolith, not microservices, unless a project is marked Enterprise and its specification says otherwise. Modules are separated so they can later be extracted.
- Every module exposes a documented internal service interface; no cross-module direct database access. • Business logic must be unit-testable without HTTP.
- All configuration by environment variable; no secrets in the repository, verified by a secret-scanning CI step. • Database migrations and seed scripts committed, idempotent, and proven to run on an empty database before every demo.
- Multi-tenant-ready from day one: every domain record carries an institution identifier. These projects are built for one university and designed to be sold to others; retrofitting tenancy costs three times as much.

TRACK J · Node.js + MongoDB. Modules are folders with an internal service interface. The domain layer must not import Express. Mongoose models belong to exactly one module.

TRACK P · Python + Django + PostgreSQL. One Django app per module – the module tables in §3 are the app list. Business logic lives in services.py, not in views: a view reads the request, calls a service and renders. Apps call each other through exposed functions, never by importing another app's models across the codebase. Money is DecimalField, never FloatField; datetimes are timezone-aware.

1.3 Non-functional baseline

Applies to both tracks unless a project states stricter numbers.

Concurrent users supported 2,000 sustained, 5,000 peak

API p95 latency Under 400 ms for reads, under 800 ms for writes Dashboard/report generation Under 3 s interactive; anything longer runs as a background job with notification

Page load (LCP) Under 2.5 s on a 4G connection

Availability target 99.5% during academic working hours Data retention 7 years for academic records; configurable per entity Backup Automated daily, tested restore documented Accessibility WCAG 2.2 AA for all student-facing screens Browser support Last two versions of Chrome, Edge, Firefox, Safari; responsive to 360 px

Localisation Internationalisation-ready (English at launch; Hindi and Punjabi strings externalised)

TRACK P · Python + Django + PostgreSQL. Django serves a fixed number of requests at a time under Gunicorn, so a slow view blocks other users. Keep views short, push slow work to Celery, and size worker counts and the database connection pool (PgBouncer) deliberately. This constraint is decisive in P14 and P19 and is called out there.

1.4 Security baseline

- OWASP Top 10 addressed and evidenced; input validation on every endpoint – Zod on Track J, Django Forms and DRF serializers on Track P
- Rate limiting per IP and per user; brute-force lockout on authentication endpoints • Encryption in transit (TLS 1.2+) and at rest; field-level encryption for identity, health and financial fields • Immutable audit log for every privileged action: actor, action, target, before and after, timestamp, IP • DPDP Act 2023 alignment: purpose limitation, consent records where required, data-subject access and deletion endpoints
- No personal data in logs, error messages or analytics events
- Dependency vulnerability scanning in CI – npm audit on Track J, pip-audit on Track P; no High or Critical findings at handover
- File uploads: MIME and magic-byte validation, size caps, malware scanning, stored outside the web root, served only via pre-signed URL

1.5 Testing requirements

Unit Jest / Vitest pytest-django ≥ 70% statement coverage on the domain and service layers

Component React Testing Library React Testing Library, or Django template tests on	Path P1 All shared components and every form
--	---

test database	Every endpoint, including
Django APITestCase / Client	authorisation-failure paths

Integration Supertest against a test database against a test database

End-to-end Playwright or Cypress Playwright or Cypress Every critical journey named in the project's acceptance criteria

Load k6 or Artillery Locust or k6 Against the concurrency targets; report submitted at M4

Security OWASP ZAP	baseline + npm audit	pip-audit
	OWASP ZAP baseline +	Clean at handover

Accessibility axe-core in CI axe-core in CI Zero critical violations on student-facing pages

Concurrency Explicit multi-client test Explicit multi-process test with TransactionTestCase	Required wherever the spec names a correctness guarantee
---	--

UAT Client-run Client-run 10 working days, defect log, severity classification

1.6 Documentation requirements

All in the repository, in Markdown: README.md with one-command local setup · architecture decision records for every significant choice · API specification · database schema documentation with an ERD · environment variable reference · deployment and rollback runbook · backup and restore procedure · role and permission matrix · admin user guide · end-user quick-start guide · a recorded handover walkthrough of 60 to 90 minutes · known issues and technical debt register.

1.7 Definition of Done (every milestone)

Code merged to main via reviewed pull request · CI green · tests written and passing · API specification updated · migrations committed and reversible · deployed to staging · demoed live · documentation updated · no High or Critical vulnerabilities · accessibility check passed on new screens.

1.8 Course mapping and permitted delivery paths

CSE326 Internet Programming	DOM, events, fetch, JSON, browser storage, debugging, Git	environment config, SEO and metadata, deployment
INT252 Web App Development with ReactJS	3 ES6+, components and props, hooks, Tailwind, controlled forms, Zod validation, Axios, React Query, routing, Context / Redux Toolkit / Zustand	Every project on both tracks. All screens, all forms, all client-side interactivity
INT253 Web Development in Python using Django	3 Models, migrations, ORM, views, URLs, templates and DTL, Django forms, CSRF, admin, users, groups, permissions, sessions, login and logout, debugging, testing	Track J front end; Track P Path P2
INT257 Modern Web App Development (Next.js)	3 App Router, SSR / SSG / ISR, server and client components, Route Handlers, Server Actions, auth and RBAC, ORM and CRUD,	The server side of every project on Track P
2 HTML5, semantic markup, forms, accessibility, CSS (flexbox, grid, responsive, animations), JavaScript,		Track J full-stack (Path J1); Track P Path P3 front end

Choose a path in week 1 and record it in the design document. Five combinations are permitted.

TRACK J · Node.js + MongoDB. Path J1 – Next.js full-stack. Route Handlers and Server Actions own the server side;

an ORM talks to the database. Covered by INT252 + INT257 + CSE326. The only Track J path fully taught by your courses.

Path J2 – React SPA + Node API. React (INT252) against an Express or NestJS API. *Gap: Express and NestJS appear in no syllabus.* Budget two to three weeks to learn the backend framework, dependency injection and OpenAPI generation.

TRACK P · Python + Django + PostgreSQL. Path P1 – Django full-stack. Django templates for pages, CSE326 for CSS and JavaScript, optionally HTMX or Alpine.js for interactivity. *Covered entirely by INT253 + CSE326.* This is the default, and the right choice for most projects here.

Path P2 – Django API + React. DRF serves JSON, React (INT252) is the front end. *Gap: Django REST Framework is not in the INT253 syllabus* – serializers, viewsets, permission classes and throttling are self-taught. Budget two weeks.

Path P3 – Django API + Next.js. As P2, with Next.js (INT257) where pages must be server-rendered or indexed by search engines – a public showcase, a public item board. *Gaps: DRF, plus the discipline of not letting two frameworks own the same logic.*

On Paths J2, P2 and P3, the server owns the database, the business rules and the permissions. The front end renders and calls the API. If a rule ends up written in both places, delete one.

1.9 What no course teaches, and which track suits which project

Everything in this list is learnable, but it is learning a team must plan for: Docker and docker compose · GitHub Actions and CI · background job systems (BullMQ or Celery) · Redis · WebSockets (Socket.IO or Channels) · object storage · load testing · production deployment – INT253 stops before it, and INT257 covers only platform deployment · Express and NestJS (Track J, Path J2) · Django REST Framework (Track P, Paths P2 and P3) · PostgreSQL constraints, locking and full-text search (Track P).

Projects where Track P is the stronger choice. P12 – the OCR and document-AI ecosystem is Python, and no separate service is needed. P13 – OR-Tools is a Python library, so the solver is an ordinary Celery task rather than a second service. P18 – the attainment arithmetic and Excel handling are natural in pandas and openpyxl. P20 – a PostgreSQL exclusion constraint makes double-booking structurally impossible. P04, P14 and P17 – the correctness guarantees these projects turn on are relational primitives.

Projects where Track J is the stronger choice. P14 and P19 at their stated concurrency targets – an event-loop server absorbs many idle connections more cheaply than synchronous workers. P16 and P09 where a statically generated public site is wanted. Any project whose team already has React depth and no Python depth.

1.10 Why Track P uses PostgreSQL and not MongoDB

Track J specifies MongoDB because it assumes a Node backend. Track P uses PostgreSQL, and is better off for it. Django's ORM, migrations, admin and permission system are built around a relational database, and the maintained Django–MongoDB adapters are not production tools. More importantly, most of these projects turn on correctness guarantees PostgreSQL provides directly:

- `transaction.atomic()` – a block of work either fully happens or fully does not
- `select_for_update()` – lock rows so two users cannot change the same thing at once
- `UniqueConstraint` – the database itself refuses to store a duplicate
- `ExclusionConstraint` – the database itself refuses to store two overlapping bookings

P04, P14, P17 and P20 are all about exactly these guarantees. On Track J a team builds them carefully by hand; on Track P the work is knowing which one to reach for and proving it holds.

2. Scale bands and weighted ranking

2.1 Scale bands and what they mean for a student team

Capacity anchor: one student contributes about 126 hours in a 14-week semester at 9 hours a week. Divide the effort estimate by team capacity before allocating. The bands are the same on both tracks; the estimates in §3 assume a team working on the track that suits the project.

Small 150–350 hrs 2–3 students, one semester Allocate whole

Medium 350–800 hrs 4–5 students, one semester up to Allocate whole up to about 600 hrs; otherwise split the about 600 hrs; two semesters above that modules Split into modules (Pattern A)

Large 800–1,800 hrs 4–5 students across two to three semesters, or two to three parallel module teams in one semester

Enterprise 1,800+ hrs Does not fit the student model as a whole system Decompose into modules, and scope the hard core as a prototype and evaluation (Pattern C)

Every specification in §3 marks which functional modules are independently buildable. Those markings are the allocation plan for Large and Enterprise projects.

2.2 Weighted ranking of all 20 projects

Criteria: educational and business value 30% · feasibility 20% · scalability as a product 20% · real-world deployment potential 20% · simplicity, meaning inverse complexity, 10%. Each scored 1–10. Column headings are abbreviated: Val = value, Fea = feasibility, Scal = product scalability, Dep = deployment potential, Sim = simplicity.

Feasibility and simplicity carry additional weight in the student model, because a project that a professional team would merely find hard is a project a student team will not finish. Read the ranking together with the band column.

1 P18 CO-PO Attainment & Accreditation Management	Large 9 7 10 10 5 8.60	Large 9 7 9 9 5 8.20
2 P15 Internship & AEDP Apprenticeship Management		
	3 P06 Placement Skill Matcher & Readiness	Large 9 7 9 9 4 8.10
4 P16 Capstone & Sponsored Project Management	Medium 8 9 8 8 7 8.10	Medium 8 8 7 8 6 7.60
5 P10 Campus Issue & Network Quality Management		
	6 P03 Hostel Allocation & Roommate Matching	Medium 8 8 7 8 6 7.60
7 P17 Student Earnings & Freelancing (EdRev) Platform	Engine	Large 8 8 8 7 6 7.60
		Enterprise 10 4 9 9 2 7.60
8 P13 Faculty Workload, Allocation & Timetable		
9 P01 Academic Risk Radar	Large 9 6 8 8 4 7.50	10 P20 Campus Resource & Lab Booking
7.50 11 P19 AI-Assisted Question Bank & Assessment	Large 8 6 9 8 4 7.40	Medium 7 9 7 7 8
12 P12 CertiScholar Document Verification & Scholarship	Large 8 6 8 8 4 7.20	
13 P14 Course Registration & Elective Allocation	Enterprise 9 5 8 8 3 7.20	14 P11 Alumni Mentorship & Career Network
Medium 7 8 7 7 6 7.10	15 P07 Health Services & Emergency Response	Medium 8 7 6 7 5 6.90
16 P05 Mess Demand & Food-Waste Management	Medium 7 8 6 7 6 6.90	17 P09 Lost & Found Management
10 5 6 10 6.70	18 P04 Hostel Room Exchange	Small 5 6 9 5 6 9 6.70
19 P02 Academic Advisor & Degree Planning	Large 8 5 8 7 3 6.70	20 P08 Research Intelligence & Collaboration Hub
		Large 7 6 7 7 4 6.50

3. Project specifications – P01 to P20

Each specification carries the same 22 sections. Sections 7, 8, 16, 18 and 19 are given separately for each track in the blue and green blocks; every other section is stack-neutral and applies to both. Sections 7, 8, 15, 16 and 17 state only the project-specific deltas from the Common Engineering Standard in section 1; everything not stated there follows the CES.

Functional-module tables mark which modules are independently buildable – those markings are the allocation plan for anything larger than one semester.

Delivery patterns referenced below.

Pattern A – module split: allocate two or three independently-buildable modules to separate teams in the same semester, with the interface contract frozen in week 2.

Pattern B – sequential semesters: successive teams, with the handover assessed as a graded deliverable.

Pattern C – prototype and evaluate: for genuinely hard components, a student team builds a working prototype with an honest written evaluation of its limits, and the institution decides later whether to fund a production build.

P01 – Academic Risk Radar

1. **Project Title** Academic Early-Warning and Mentoring Intervention Platform (Academic Risk Radar). *Track J:* React/Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. **Project Overview** *Description.* A platform that consolidates attendance, assessment, assignment and LMS-engagement signals into a single student-level view, detects meaningful deterioration against each student's own baseline, and routes explained alerts to the assigned faculty mentor, who records the intervention and its outcome. *Problem.* Academic difficulty is currently discovered when results are declared. The signals that predict it – falling attendance, declining internal marks, missed submissions, reduced LMS activity – exist in four separate systems and are never combined. No one owns the moment of noticing. *Target users.* Students, course instructors, faculty mentors, heads of department, Dean of Academics, system administrators. *Expected outcome.* Alerts raised 4–8 weeks earlier than current practice, every alert traceable to a recorded mentoring action and a measured follow-up.

Governance constraint that must be built in, not bolted on: this is an early-support system, never a disciplinary one. Risk bands are visible to the student themselves with their contributing factors; they are never exposed to recruiters, never printed on any transcript, and never used as an eligibility filter anywhere in the system. Build that as an access-control rule, not a policy note.

3. Project Objectives

- Ingest and normalise attendance, marks, assignment and LMS-activity data from existing institutional sources
- Produce a per-student risk band with explicit, ranked contributing factors – no unexplained score
- Detect trend deterioration against the student's own rolling baseline, not only against cohort thresholds
- Route alerts to the correct mentor with a defined acknowledgement SLA
- Capture intervention records and follow-up outcomes as structured data
- Provide course, department and institution-level analytics on risk patterns and mentoring effectiveness
- Maintain a complete audit trail of thresholds, weights, alerts and actions

4. **Key Features** Unified student academic profile · configurable risk model with versioned weights and thresholds · factor attribution on every alert · sudden-drop and trend detection · mentor alert inbox with acknowledgement SLA · intervention logging and outcome follow-up · cohort and department dashboards · mentoring-effectiveness reporting · full audit log · scheduled recomputation with manual override.

5. Functional Modules

M1 Data Ingestion & Normalisation	Consolidated per-student view: enrolment, sections, history, current standing	and Dean views with role-scoped data Yes – build first
M2 Student Academic Profile	Versioned rule and weight engine	
M3 Risk Model & Scoring Engine	producing a band plus ranked factor attribution; runs as a queued batch job	Yes
M4 Trend & Anomaly Detection		
M5 Alert Routing & Mentor Inbox	Rolling-baseline comparison per student; sudden-drop detection	Depends on M1, M2
M6 Intervention & Outcome Tracking	Mentor mapping, alert generation, deduplication, acknowledgement SLA, escalation	Depends on M1 Depends on M3,
M7 Dashboards & Reporting	Structured intervention records, follow-up scheduling, outcome capture	M4 Yes – parallel build Depends on
Connectors and importers for attendance, marks, assignments, LMS activity; scheduled sync; validation and quarantine of bad records	Student, instructor, mentor, HoD	M2, M3

M8 Configuration & Audit Threshold and weight administration with versioning;

Yes

immutable audit log

6. **User Roles & Permissions** Student – own profile, own risk band, own contributing factors, own recommended actions; no access to any other student's data. Course Instructor – academic indicators for students in their own sections only. Faculty Mentor – full profile and risk detail for assigned mentees; create interventions; record outcomes. Head of Department – department-level aggregates, course concentrations, mentoring activity; individual detail only for their department. Dean of Academics – institution-level aggregates, audit reports; individual detail read-only. System Administrator – thresholds, weights, connectors, follow-up settings; no access to individual assessment data, enforced in the authorisation layer.

7. Technology Stack

TRACK J · Node.js + MongoDB. BullMQ for nightly scoring jobs; Recharts for dashboards; scheduling via BullMQ repeatable jobs, never in-process cron. An optional Python microservice for the scoring model only if the team justifies it; a rules-plus-weights engine in TypeScript is sufficient for v1 and is preferred.

TRACK P · Python + Django + PostgreSQL. Path P1. Celery + Celery Beat for nightly scoring; Chart.js in templates for dashboards; pandas for the import step; django-debug-toolbar in development to watch the query count. The scoring engine is plain Python in services.py with no Django imports, so it is unit-testable in isolation. Do not reach for scikit-learn: a weighted rules engine is what v1 needs.

8. **Technical Requirements (deltas from CES)** *Frontend (both tracks):* role-scoped routing; factor-attribution visualisation; mentor inbox with bulk acknowledgement; student view deliberately designed to be non-alarming – copy and colour tested with the client. *Data model (both tracks):* Student, Enrolment, Course, Section, AttendanceRecord, AssessmentRecord, AssignmentSubmission, LmsActivity, RiskSnapshot, RiskModelVersion, Alert, Intervention, MentorAssignment, AuditEntry. RiskSnapshot is append-only and time-series, indexed on student plus computed-at. *API (both tracks):* /api/v1/students/:id/risk, /alerts, /interventions, /analytics/*, /admin/model-versions. Every analytics endpoint paginated and role-filtered server-side. *Performance (both tracks):* idempotent nightly batch scoring across 30,000+ students within 30 minutes; recomputation triggerable per cohort; dashboards under 3 s at department scale; the scoring job isolated from the request-serving process. *Security (both tracks):* field-level access control tested explicitly – the integration suite must include a case asserting that a student cannot read another student's risk record and that an administrator cannot read raw marks.

TRACK J · Node.js + MongoDB. MongoDB aggregation pipelines carry most of this project; \$lookup joins across the four source collections, \$setWindowFields for rolling baselines. The connector framework must tolerate partial source availability. Scoring workers deployed separately from the API.

TRACK P · Python + Django + PostgreSQL. The ORM carries most of this project: annotate() with Avg and Count, and window functions with Window and Frame for rolling baselines. Import with pandas then bulk_create in batches. Watch the query count – the naive per-student loop issues 30,000 queries; select_related and prefetch_related are not optional here. Quarantined rows go to a separate model, never dropped.

9. **Admin Dashboard** Model version management with effective dates and rollback · threshold and weight editor with a preview showing how many students would change band · connector health and last-sync status · alert volume and acknowledgement SLA compliance · user and role administration · audit log viewer with filters and export.

10. **Reporting & Analytics** Risk distribution by course, programme, semester and department · trend over time · mentor response-time and closure-rate reporting · intervention-outcome analysis, meaning whether the band improved after action · false-positive review queue · exports to CSV and PDF · scheduled email reports to HoDs.

11. **Notifications & Communication** In-app inbox · email through a transactional provider · optional SMS for high-severity only · digest preferences per user · escalation to HoD when an alert is unacknowledged beyond SLA · notification templates editable by administrators.

12. **Third-Party Integrations** University ERP/SIS (read-only), LMS (Moodle, Canvas or institutional), email provider, SSO/LDAP. All integrations sit behind an adapter interface with a working file-import fallback – assume at least one source system will not be available on schedule.

13. **Expected Deliverables** Source in the client GitHub organisation · deployed staging and production environments · API specification · seeded demo dataset · admin and user guides · architecture decision records · test suites · load-test report · recorded handover · 30-day defect-fix window.

14. **Development Milestones** M0 discovery, data model, API contract, wireframes (2w) · M1 M1+M2 ingestion and profile, auth, CI/CD, staging (3w) · M2 M3+M4 scoring and trend detection with explainability (4w) · M3 M5+M6 alerts, mentor inbox, interventions (3w) · M4 M7+M8 dashboards, admin, audit, reports (4w) · M5 performance, UAT, documentation, production (2w).

15. **Testing** Per CES. Project-specific end-to-end journeys: student views own risk; mentor receives, acknowledges and closes an alert; HoD runs a department report; administrator changes a weight and sees the preview; a student attempts to access another student's record and is denied. Scoring must be verified reproducible: the same inputs and model version produce the same bands.

16. **Deployment** Staging from M1; production behind university SSO.

TRACK J · Node.js + MongoDB. Batch workers deployed as a separate service from the API, scaled independently.

TRACK P · Python + Django + PostgreSQL. Celery workers run as separate containers from Gunicorn, with Celery Beat as a single scheduler instance. Never run Beat in more than one replica.

17. **Documentation** Per CES, plus a written model specification: every factor, its weight, its source field, and the exact definition of each band. This document is what the Academic Council will ask for.

18. **Skills Students Need**

TRACK J · Node.js + MongoDB. TypeScript · React/Next.js · Node.js or NestJS · MongoDB aggregation pipelines – most of this project is aggregation · BullMQ and Redis · data integration · RBAC design · charting · Docker · GitHub Actions.

TRACK P · Python + Django + PostgreSQL. Python · Django ORM at depth, including aggregation and window functions – most of this project is aggregation · Celery · pandas · PostgreSQL indexing · Django permissions · CSE326 for dashboards and charts · Docker · GitHub Actions.

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One team member should have taken a statistics or data-analytics elective. *Track J:* strong with MongoDB aggregation. *Track P:* INT253 complete and comfortable enough with the ORM to reason about the queries it generates; gap to plan for: Celery and window functions.

20. **Estimated Complexity and Semester Fit** Large. 1,100–1,500 hours. Two semesters for a team of 5, or three parallel module teams in one semester (M1+M2 ingestion and profile · M3+M4 scoring and trends · M5+M6+M7 alerts, interventions and dashboards).

21. **Suggested Student Team** 5 students – tech lead · 2 backend, one owning the aggregation work · frontend and data visualisation · QA, documentation and integration.

22. **Acceptance Criteria**

- Ingests all four data sources and quarantines malformed records without failing the batch
- Nightly scoring completes for 30,000 students in under 30 minutes

- Every alert displays at least three ranked contributing factors traceable to source records
- Mentor acknowledgement SLA and escalation demonstrably work
- Role isolation proven by the authorisation test suite, including negative cases
- Dashboards render under 3 s at department scale
- Model weights changeable without a code deploy, with version history and rollback
- Full audit trail exportable

- CES non-functional baseline met and evidenced by the load-test report

AI/ML potential: v1 must be a transparent weighted-rules engine. Once 2–3 semesters of labelled outcomes exist, a gradient-boosted classifier with SHAP attribution can replace the scoring while preserving explainability. Do not start with ML – you have no labels and no trust.

Commercialisation: High. Every institution with an NBA or NAAC mentoring obligation needs this. Multi-tenant from day one.

P02 – Academic Advisor and Degree Planning System

1. Project Title Curriculum Rules Engine and Degree Planning Platform with Advisor Workflow. *Track J:* Next.js + NestJS + MongoDB. *Track P:* Django + PostgreSQL.
2. Project Overview *Description.* A system that encodes curriculum structure and prerequisite rules in

machine-readable form, shows each student their degree progress and eligibility with reasons, lets them simulate future semester plans without touching official records, and routes a chosen plan through faculty advisor approval with versioning. *Problem.* Students choose courses without a reliable view of prerequisites, remaining requirements or graduation impact. Advisors approve plans from memory and spreadsheets. Errors surface a semester later as delayed graduation. *Target users.* Students, faculty advisors, programme coordinators, HoDs, administrators. *Expected outcome.* Fewer prerequisite violations, fewer delayed graduations, advising conversations that start from evidence.

3. Project Objectives

- Model programmes, curricula, requirement groups, credit categories and prerequisite logic as data, not code
- Compute degree progress and course eligibility with a human-readable explanation for every ineligibility
- Support what-if plan simulation across multiple future semesters without mutating records
- Provide a controlled advisor review, approval, override and versioning workflow
- Relate course choices to declared career objectives with explained recommendations
- Keep faculty judgement as the final authority – recommendations are advisory and every override is recorded with justification

4. Key Features Curriculum and prerequisite authoring · degree audit and progress tracker · eligibility engine with reasons · multi-semester plan simulator · career-objective mapping · skill-gap and co-curricular suggestions · advisor review queue with comments · plan versioning and diff · override justification capture · programme-level advising analytics.

5. Functional Modules

M1 Curriculum & Requirement Authoring	Programmes, versions, requirement groups, credit categories, course catalogue	Yes – build first
M2 Prerequisite Rule Engine	Boolean, credit, CGPA, co-requisite and anti-requisite logic; evaluation with reason output	Yes – the core IP; built and unit-tested standalone
M3 Student Academic Record	Transcript, completed and in-progress credits, exemptions, transfers	Yes
M4 Degree Progress Audit	Maps records against requirement groups; computes remaining	Depends on M1, M3
M5 Eligibility Checker	Per-course eligibility with reasons	Depends on M2, M3
M6 Plan Simulator	Sandboxed multi-semester planning, offering-aware, graduation-date projection	approval, override with justification, versioning Depends on M4, M5
M7 Career & Skill Mapping	Objective to course and skill mapping; gap identification	Yes – data-driven, parallel build
M8 Advisor Workflow	Submission, review, comments, Advising throughput, common blockers, plan-approval status	Depends on M6 Depends on M8
M9 Analytics & Administration		

6. User Roles & Permissions Student – own record, own audit, create, simulate and submit own plans. Faculty Advisor – assigned advisees' records and plans; comment, approve, return, override with justification. Programme Coordinator – curriculum authoring, prerequisites, credit categories, course offerings. Head

of Department – department advising status and analytics. System Administrator – recommendation configuration, career-objective taxonomy, user administration.

7. Technology Stack

TRACK J · Node.js + MongoDB. NestJS strongly preferred: this project has the most complex domain logic in the portfolio and benefits from its module and dependency-injection structure. Rule evaluation in pure TypeScript over an interpreted rule AST. Drag-and-drop planner with dnd-kit.

TRACK P · Python + Django + PostgreSQL. Rules stored as structured data in a JSONB field, evaluated by a small interpreter written in plain Python with no Django imports – which is what makes it properly unit-testable. Curriculum versions immutable once published: a published flag plus a save() guard, with each student pinned to a version by foreign key. Path P2 is reasonable here because the planner is genuinely interactive; Path P1 works if the planner is a form-driven page.

On both tracks: never evaluate prerequisite rules with eval, exec or a scripting sandbox. Rules are data, interpreted by code you wrote. Say this in the posting.

8. Technical Requirements (deltas) Frontend (both): drag-and-drop semester planner; timeline visualisation of the graduation projection; side-by-side plan version diff; requirement-group progress components. **Data model (both):** Programme, CurriculumVersion, RequirementGroup, Course, PrerequisiteRule, CourseOffering, StudentRecord, TranscriptEntry, Plan, PlanVersion, AdvisorAssignment, CareerObjective, SkillMapping, Override, AuditEntry. **API (both):** /curriculum/*, /students/:id/audit, /students/:id/eligibility, /plans, /plans/:id/simulate, /plans/:id/submit, /advisor/queue. **Performance (both):** a full audit for one student in under 200 ms; batch audit of a 5,000-student cohort as a background job with progress reporting. **Security (both):** advisors scoped to assigned advisees; overrides require justification text and are immutable once recorded.

TRACK J · Node.js + MongoDB. Versioned hierarchical curriculum data modelled as embedded documents where read together and referenced where shared. Cohort pinning by curriculum version identifier on the student record.

TRACK P · Python + Django + PostgreSQL. Relational modelling suits this domain unusually well: requirement groups are a self-referencing tree, prerequisites are rows, and a recursive CTE (raw SQL through the ORM) answers "what does this course transitively require" in one query. Override immutability enforced with a save() guard and a database-level check.

9. Admin Dashboard Curriculum version manager with publish and pin controls · prerequisite rule builder with a test harness, where a hypothetical student is entered and the evaluation shown · offering management per term · advisor assignment · override audit · advising SLA monitoring.

10. Reporting & Analytics Cohort graduation-risk projection · most common blocking prerequisites · elective demand forecasting, which directly feeds P14 and P13 · advisor workload and turnaround · plan approval funnel · exportable programme audit reports for accreditation.

11. Notifications & Communication Plan submitted, returned or approved · registration window reminders · prerequisite-risk warnings before registration opens · advisor queue digests.

12. Third-Party Integrations ERP/SIS for transcripts (read), course offering feed, SSO. Publishes an eligibility API consumed by P14.

13. Expected Deliverables Per CES, plus a rule-authoring guide for programme coordinators and a seeded curriculum for at least two real programmes.

14. Development Milestones M0 discovery, curriculum modelling workshop, rule taxonomy (3w) · M1 M1+M2 authoring and rule engine with test harness (4w) · M2 M3+M4+M5 records, audit, eligibility (4w) · M3 M6 plan simulator (4w) · M4 M7+M8 career mapping and advisor workflow (4w) · M5 M9 analytics, UAT, documentation, production (3w).

15. Testing Per CES. Rule-engine unit coverage must be $\geq 90\%$ with a fixture library of at least 60 real prerequisite scenarios supplied by the client. End-to-end: student builds and submits a plan; advisor returns it; student revises; advisor approves; override recorded.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. Batch audit workers separate from the API.

TRACK P · Python + Django + PostgreSQL. Per CES. Batch audit as a Celery task; the rule interpreter is pure Python and needs no special deployment.

17. Documentation Per CES. Documentation must include the complete rule grammar specification – what a rule may express, how it is written, and how it is evaluated.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · NestJS · complex domain modelling · rules and constraint engines · React with dnd-kit · MongoDB modelling of versioned hierarchical data · data visualisation.

TRACK P · Python + Django + PostgreSQL. Python · writing a small interpreter · Django ORM with self-referencing trees and recursive queries · JSONB · complex domain modelling · CSE326 or INT252 for the planner UI.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member must be genuinely strong at abstract modelling – this project lives or dies on the rule engine. Prior exposure to compilers, formal methods or discrete mathematics is a real advantage on either track. *Track P gap:* writing an interpreter is taught in no course on this list.

20. Estimated Complexity and Semester Fit Large, at the upper boundary. 1,400–1,800 hours. The hardest non-Enterprise project in the portfolio. Allocate as three module teams (M1+M2 authoring and rule engine · M3+M4+M5 records, audit and eligibility · M6+M8 simulator and advisor workflow) across two semesters. Do not give it to a first-time team.

21. Suggested Student Team 5 students per module team – tech lead · 2 backend, one dedicated to the rule engine · frontend · QA and documentation.

22. Acceptance Criteria

- Curriculum for two real programmes authored entirely through the UI with no code changes
- All 60 client-supplied prerequisite scenarios evaluate correctly with correct reasons
- Full degree audit for one student in under 200 ms
- Plan simulation shows graduation-date impact and flags infeasible plans with the blocking reason
- Curriculum version pinning verified: a student on the 2024 curriculum is unaffected by a 2026 change
- Every advisor override carries a stored justification and appears in the audit report
- Batch audit of 5,000 students completes as a background job with progress reporting

AI/ML potential: Elective recommendation from historical co-enrolment and outcome data; graduation-delay prediction. Only after two years of plan data.

Commercialisation: High but heavy – degree-audit systems are a mature commercial category (DegreeWorks and similar), so differentiation must come from the simulator experience and price.

P03 – Hostel Allocation and Roommate Matching

1. Project Title Policy-Driven Hostel Allocation Engine with Roommate Compatibility Matching. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. Project Overview *Description.* Digitises hostel inventory and the allocation cycle: captures applications and ranked preferences, validates eligibility against approved policy, generates a draft allocation honouring capacity, quota and accessibility constraints, and routes the draft to wardens for documented review before publication. *Problem.* Allocation for thousands of applicants is run on spreadsheets against several simultaneous rules – capacity, quota, fee category, programme conditions, accessibility. The process is slow, hard to explain when challenged, and produces no reviewable record. *Target users.* Students, wardens, chief warden, hostel administrators, Dean of Student Welfare, administrators. *Expected outcome.* An allocation cycle completed in days rather than weeks, with every decision explainable and auditable.

Governance constraint: the engine produces a draft. Publication requires warden review. Build the draft/published state separation into the data model – do not allow a code path that publishes an allocation without an approval record.

3. Project Objectives

- Maintain hostel, block, floor, room and bed inventory with accessibility attributes
- Manage application cycles with windows, eligibility rules and document requirements •
- Capture ranked preferences and consented lifestyle attributes for compatibility matching •
- Generate a constraint-respecting draft allocation with an explanation per assignment •
- Support documented warden review, override and waiting-list management
- Publish the final allocation, generate allocation letters and retain the full history

4. Key Features Inventory management with bed-level granularity · cycle configuration · eligibility validation with reasons · ranked preference capture · consented lifestyle questionnaire · allocation engine with hard and soft constraints · draft review with per-assignment override and reason · waiting list · publication and letter generation · occupancy dashboard · complete audit.

5. Functional Modules

M1 Hostel & Room Inventory	documents, submission	Yes	Rule-based eligibility
M2 Application & Cycle Management		with reason output	Depends on M2
M3 Eligibility & Policy Validation			
Hostels, blocks, floors, rooms, beds, attributes,			
status	Yes – build first	Cycles, windows, forms,	
M4 Preference Management	Ranked hostel, room-type and roommate preferences	Depends on M1, M2	
M5 Compatibility	Questionnaire & Scoring	pairwise compatibility score	
	Consented lifestyle attributes; Yes	(preference, compatibility)	
M6 Allocation Engine	Hard constraints (capacity, quota, accessibility, gender block) and soft constraints	Depends on M1, M3, M4, M5	
M7 Warden Review & Override	Draft review interface, per-bed reassignment, mandatory reason,	approval	Depends on M6
M8 Waiting List	Ordered list, automatic promotion on vacancy	Depends on M6	
M9 Publication, Letters & Audit		history	Depends on M7
Publish, PDF letters, notifications, immutable			

6. User Roles & Permissions Student – own application, preferences, questionnaire, allocation result, room-change request. Warden – draft review for own hostel, override with reason, waiting list, publish own hostel.

Chief Warden – cross-hostel review, escalated approvals, publish all. Hostel Administrator – inventory, cycles, quotas, policy configuration. Dean of Student Welfare – occupancy, allocation and override analytics, read-only. System Administrator – allocation priorities, compatibility weights, user administration.

7. Technology Stack

TRACK J · Node.js + MongoDB. Allocation engine as a BullMQ background job; deterministic greedy or heuristic assignment with a seeded random tiebreak, the algorithm stated and justified in the week-2 design review. PDF letters via a server-side renderer. An optional Python/OR-Tools microservice only if the team can justify it against the deterministic approach.

TRACK P · Python + Django + PostgreSQL. Allocation as a Celery task with progress on an AllocationRun row. Seed the tiebreak with random.Random(seed) and store the seed, so a warden can re-run and get an identical draft. Bed state changes inside transaction.atomic(). Letters with WeasyPrint – an HTML template with CSS, which is exactly what CSE326 taught. OR-Tools is available in-process if the team wants a true optimiser, but a documented heuristic is preferred for v1.

8. Technical Requirements (deltas) *Frontend (both):* visual bed map per floor with drag-to-reassign in the warden view; student preference ranking interface; mobile-first student application. *Data model (both):* Hostel, Block, Room, Bed, AllocationCycle, Application, Preference, CompatibilityResponse, EligibilityRule, AllocationDraft, AllocationAssignment, Override, WaitlistEntry, AllocationLetter, AuditEntry. *API (both):* /inventory/*, /cycles/:id/applications, /cycles/:id/allocate, /drafts/:id, /drafts/:id/override, /drafts/:id/publish, /waitlist. *Performance (both):* allocation for 8,000 applicants against 8,000 beds within 10 minutes, deterministic and reproducible from a stored seed; draft immutability once published. *Security (both):* lifestyle questionnaire data is sensitive – encrypted at rest, visible only inside aggregate compatibility scores, never shown to other students, deletable on request.

TRACK J · Node.js + MongoDB. Bed assignment writes wrapped in a MongoDB transaction so a partial draft can never be persisted.

TRACK P · Python + Django + PostgreSQL. A UniqueConstraint on the assignment's bed within a draft makes double-assignment impossible at the database level rather than by convention. Encrypt questionnaire fields with django-cryptography or pgcrypto.

9. Admin Dashboard Inventory CRUD with bulk import · cycle configuration · quota and priority rules · allocation run history with parameters and seed · occupancy heatmap · override report by warden · document verification queue.

10. Reporting & Analytics Occupancy by hostel, block and room type · preference satisfaction rate, meaning what percentage received their first choice · override frequency and reasons by warden · waiting-list movement · allocation cycle time · accessibility compliance report · year-on-year comparison.

11. Notifications & Communication Application window open and close, document required, draft published, allocation result, waiting-list promotion, room-change status. Email plus in-app; SMS for the final allocation.

12. Third-Party Integrations ERP/SIS for enrolment and fee-category verification, document storage, email and SMS, SSO. Publishes an occupancy API consumed by P04.

13. Expected Deliverables Per CES, plus allocation-letter templates and a documented algorithm specification.

14. Development Milestones M0 discovery and policy capture (2w) · M1 inventory and applications (3w) · M2 eligibility, preferences, questionnaire (2w) · M3 allocation engine (3w) · M4 warden review, waitlist, publication (3w) · M5 analytics, UAT, production (2w).

15. Testing Per CES. The allocation engine must have a property-based test suite: no student assigned two beds, no bed double-assigned, no hard constraint violated, reproducibility from seed. End-to-end: apply, fail eligibility with reason, correct, allocate, warden override, publish, student sees result.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. Allocation workers separate from the API so a 10-minute run never blocks requests.

TRACK P · Python + Django + PostgreSQL. Per CES. Celery worker container sized for the allocation run; the web container never executes it.

17. Documentation Per CES, plus a written policy-to-rule mapping document signed off by the Hostel Administration.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Node.js · MongoDB · constraint and assignment algorithms · complex form UX · PDF generation · bulk data import.

TRACK P · Python + Django + PostgreSQL. Python · Django models and forms · constraint and assignment algorithms · Celery · WeasyPrint · CSE326 for the bed map · bulk import with `bulk_create`.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with algorithms and constraint reasoning; one comfortable with complex multi-step form UX. *Track P gaps:* Celery, WeasyPrint, and the allocation algorithm itself.

20. Estimated Complexity and Semester Fit Medium, upper end. 600–800 hours. Two semesters for a team of 4, or one semester for a team of 5 if the allocation engine (M6) is scoped as a documented heuristic rather than an optimiser.

21. Suggested Student Team 5 students – tech lead · 2 backend, one owning the allocation engine · frontend · QA and documentation.

22. Acceptance Criteria

- 8,000 applications allocated in under 10 minutes, reproducibly from a stored seed
- Zero hard-constraint violations across the property-based test suite
- Every assignment carries an explanation; every override carries a mandatory reason
- No allocation can be published without a recorded approval – proven by a negative test
- Preference-satisfaction and override reports generated for a full cycle
- Lifestyle data never exposed at individual level to any student

AI/ML potential: Learned compatibility weighting from historical room-change requests – a genuinely useful second-year feature, since room-change frequency is a direct label for match quality.

Commercialisation: Strong for the Indian residential-university market, where this is a universal and universally painful problem.

P04 – Hostel Room Exchange

1. **Project Title** Hostel Room Exchange and Multi-Party Swap Platform. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. **Project Overview** *Description.* A formal channel for post-allocation room changes: students post exchange requests, the system validates them against policy and detects both direct swaps and multi-party rotation chains, all affected students must accept, an authorised warden approves, and occupancy records update automatically with a transfer record. *Problem.* Room changes happen informally through WhatsApp and warden desks. Occupancy records drift out of date, policy checks are skipped, and there is no record of who moved when or why. *Target users.* Students, wardens, chief warden, hostel administrators, Dean of Student Welfare. *Expected outcome.* Every room change flows through one auditable channel, and occupancy records stay accurate.

3. Project Objectives

- Provide a single official channel for exchange requests
- Validate eligibility – cooling-off period, fee status, disciplinary holds, gender-block rules – before matching
- Detect direct swaps and cycles of length 3–5
- Require acceptance from every affected student and approval from the responsible authority
- Update occupancy atomically and generate a transfer record

4. **Key Features** Exchange request creation with target preferences · policy validation with reasons · direct-swap and cycle detection · multi-party acceptance workflow with expiry · warden approval · atomic occupancy update · transfer record generation · cooling-off enforcement · request history.

5. Functional Modules

M1 Exchange Request Management	detection over the request graph
M2 Eligibility & Policy Validation	M4 Multi-Party Acceptance Proposal
Create, modify, withdraw, expire requests Yes	notification, time-boxed acceptance,
	all-or-nothing
Cooling-off, holds, fee status, block rules Yes	Depends on M1 Depends on M3
M3 Matching Engine Direct swaps and cycle	
M5 Approval Workflow Warden and chief warden approval with remarks	Depends on M4
M6 Occupancy Update & Transfer Record	M5
Atomic transaction, letter generation	Depends on

6. **User Roles & Permissions** Student – own requests, accept or decline proposals, view own transfer record. Warden – review and approve exchanges within own hostel. Chief Warden – cross-hostel exchanges. Hostel Administrator – eligibility rules, cooling-off and validity settings. Dean of Student Welfare – analytics, read-only.

7. Technology Stack

TRACK J · Node.js + MongoDB. Cycle detection is a graph problem: implement in TypeScript over an in-memory adjacency structure. MongoDB multi-document transactions are mandatory for the occupancy update – this is the one place in the project where correctness is non-negotiable, and it must be stated in the posting. Note that this requires a replica set even in development.

TRACK P · Python + Django + PostgreSQL. Cycle detection with networkx, or thirty lines of plain Python. The occupancy update uses `transaction.atomic()` with `select_for_update()`, locking every bed in the chain in a fixed order – sort by primary key – which is how you avoid a deadlock when two wardens approve overlapping chains

in the same second. Proposal expiry is a Celery Beat sweep over an `expires_at` column, never a timer held in memory.

8. **Technical Requirements** (*deltas*) *Backend (both)*: cycle detection bounded to length 5, running in under 5 s over 2,000 open requests; proposals expire automatically; acceptance is all-or-nothing; concurrent approval safety – two wardens approving overlapping chains must not corrupt occupancy. *Data model (both)*: ExchangeRequest, ExchangeProposal, ProposalParticipant, Approval, TransferRecord, PolicyRule, AuditEntry. Bed and occupancy state is read from P03's data or through its API. *API (both)*: `/exchange-requests`, `/proposals`, `/proposals/:id/respond`, `/proposals/:id/approve`, `/transfers`.

TRACK J · Node.js + MongoDB. The transaction must cover every participant's bed change; a partially applied rotation is the failure mode to design against.

TRACK P · Python + Django + PostgreSQL. PostgreSQL gives this project its teaching value cheaply: the correctness requirement is one atomic block, one ordered `select_for_update`, and a test that proves it. Write the concurrency test with `TransactionTestCase` and real threads – `TestCase` wraps each test in a transaction and will hide the bug you are looking for.

9. **Admin Dashboard** Policy configuration – cooling-off, validity, maximum chain length · open-request pool · proposal pipeline · approval queue · exchange volume and pattern reports.

10. **Reporting & Analytics** Request volume by hostel and period, match rate, average time to completion, approval and rejection patterns by warden, most-requested rooms – a useful signal back into P03's compatibility weighting.

11. **Notifications & Communication** Proposal received with expiry countdown, participant accepted or declined, approval granted, transfer completed. Time-sensitive: push and email.

12. **Third-Party Integrations** P03 occupancy (mandatory), ERP for fee and disciplinary holds, email and SMS, SSO.

13. **Expected Deliverables** Per CES.

14. **Development Milestones** M0 discovery (1w) · M1 requests and validation (2w) · M2 matching engine (2w) · M3 acceptance and approval (2w) · M4 occupancy update, records, reports (1w) · M5 UAT and production (1w).

15. **Testing** Per CES. Critical cases: 3-way and 4-way cycles; partial acceptance must not execute; concurrent approvals of overlapping chains; proposal expiry; occupancy consistency asserted after every transfer.

16. **Deployment**

TRACK J · Node.js + MongoDB. Per CES. MongoDB must run as a replica set in every environment, including local

development, or transactions are unavailable – plan for this in the compose file.

TRACK P · Python + Django + PostgreSQL. Per CES. No special deployment need; a single PostgreSQL instance gives full transactional semantics.

17. Documentation Per CES, plus a written note on the locking strategy and why it is correct – this is the artefact that shows the team understood the problem.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Node.js · MongoDB transactions · graph algorithms · workflow and state-machine design · React.

TRACK P · Python + Django + PostgreSQL. Python · PostgreSQL transactions and row locking · graph algorithms · Django models and state-machine design · CSE326 for the interface · Celery for expiry.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member must understand transactional correctness and be willing to think carefully about concurrency. This is the teaching point of the project on either track. *Track*

P note: everything else here sits inside INT253 and CSE326, which is what makes this the best first project on that track.

20. Estimated Complexity and Semester Fit Small, upper end. 280–350 hours. One semester for a team of 3.

21. Suggested Student Team 3 students – tech lead, who owns the swap transaction · full-stack · frontend, QA and documentation.

22. Acceptance Criteria

- 4-party rotation detected, proposed, accepted and executed end to end
- Partial acceptance never mutates occupancy – proven by an explicit test
- Concurrent approval of overlapping chains cannot corrupt occupancy – proven by a concurrency test
- Occupancy in P03 reconciles exactly after 100 simulated transfers
- Cooling-off and policy rules configurable without deployment
- Complete audit trail per transfer

Note on allocating this one. P04 is an excellent semester-1 project. It is small and unambiguous, and it contains exactly one genuinely hard correctness requirement – the transactional multi-party swap – which is the sort of problem students rarely meet in coursework and remember afterwards. A team that gets this right has earned a Large module next semester.

Commercialisation: Low standalone; high as a bundled module alongside P03.

P05 – Mess Demand Forecasting and Food-Waste Management

1. **Project Title** Meal Demand Forecasting and Food-Waste Reduction Platform. *Track J:* Next.js + Node.js + MongoDB, plus an optional Python forecasting service. *Track P:* Django + PostgreSQL, forecasting in-process.

2. **Project Overview** *Description.* Forecasts attendance for each meal service using historical attendance and calendar context, converts the forecast into preparation and ingredient quantities via recipe data, captures post-service actuals – prepared, served, leftover – and reports waste quantity and cost with forecast-accuracy feedback. *Problem.* Kitchens prepare against registered strength, not actual demand. Demand varies with menu, weekday, holidays, examinations and events. Over-preparation is routine, waste is unmeasured, and its cost is invisible. *Target users.* Students, kitchen staff, mess managers, mess administrators, finance officers. *Expected outcome.* Measurable reduction in avoidable waste, a documented cost saving, and a planning loop that improves each cycle.

3. Project Objectives

- Maintain menus, dishes, recipes, portion sizes and ingredient mappings
- Capture meal attendance reliably per service
- Forecast attendance per meal using history plus calendar features
- Convert the forecast to preparation quantities and an ingredient indent
- Capture post-service actuals and compute waste quantity and cost
- Report forecast accuracy and high-waste dishes to drive menu and process change

4. Key Features Menu and recipe management with portion and ingredient mapping · attendance capture by QR, RFID or manual entry · calendar and event registry · forecasting engine with confidence range · service-level configuration, meaning how much over-preparation is acceptable · preparation guidance and ingredient indent generation · post-service actuals capture on a mobile-friendly kitchen screen · waste costing · forecast-accuracy tracking · high-waste dish reporting.

5. Functional Modules

M1 Menu, Dish & Recipe Management	Dishes, recipes, portions, ingredients, costs, dining halls	exam periods, campus events Yes – build first
M2 Meal Attendance Capture	Per-service attendance via scan or manual entry; offline-tolerant	Yes
M3 Calendar & Event Registry	Academic calendar, holidays, Aggregation by meal, weekday, dish, period	
M4 Historical Demand Analysis	Depends on M2	
M5 Forecasting Engine Attendance prediction with confidence interval	Depends on M3, M4	
M6 Preparation & Ingredient Planning	M1, M5	
Forecast to portions to ingredient indent	Depends on	
M7 Post-Service Recording Prepared, served and leftover capture per dish	Depends on M1	
M8 Waste Measurement & Costing	Forecast against actual, high-waste dishes, savings	
M9 Accuracy & Improvement Reporting	Depends on M5, M8	
Quantity and cost of waste; trend	Depends on M7	

6. User Roles & Permissions Student – menu view, own meal attendance, optional feedback. Kitchen Staff – preparation guidance, actuals entry for the assigned hall and service. Mess Manager – menu planning, forecast review, service-level settings, waste analysis. Mess Administrator – dishes, recipes, ingredients, halls. Finance Officer – waste cost, procurement summaries, read-only. Administrator – forecast configuration.

7. Technology Stack

TRACK J · Node.js + MongoDB. Start with a statistical baseline in TypeScript – moving average with weekday and event adjustment – and introduce a Python service (Prophet or gradient boosting) only if the baseline’s error exceeds an agreed threshold after one term of data. The week-2 design review must record this staged approach.

TRACK P · Python + Django + PostgreSQL. Python is an advantage here. The baseline is a few lines of pandas inside a Celery task, and the upgrade path to statsmodels or scikit-learn needs no second service – but the same staged rule applies: baseline first, learned model only after a term of real data.

On both tracks: a team that reaches for machine learning on a problem with no data yet has not understood the problem. The baseline is the deliverable for semester 1.

8. Technical Requirements (deltas) *Frontend (both)*: kitchen screens must be tablet-first, high-contrast and glove-friendly, with large touch targets and offline capability – this is a wet, hot environment, not a desk. *Offline sync (both)*: a browser service worker with IndexedDB queues scans locally and syncs when connectivity returns. The sync endpoint must be idempotent – every offline record carries a client-generated UUID and the server ignores one it has already stored. *Data model (both)*: Dish, Recipe, Ingredient, Menu, MenuSchedule, DiningHall, MealService, AttendanceRecord, CalendarEvent, Forecast, PreparationPlan, ServiceActual, WasteRecord, CostRate. *API (both)*: /menus, /attendance, /forecasts, /preparation-plans, /actuals,

/analytics/waste. *Performance (both)*: attendance capture must handle 3,000 scans in a 45-minute service window across multiple halls; a 7-day forecast generated daily; the ingredient indent exportable in the procurement format in use.

TRACK J · Node.js + MongoDB. Time-series aggregation with MongoDB's time-series collections; conflict resolution on sync handled in the service layer.

TRACK P · Python + Django + PostgreSQL. Aggregate with TruncDate and annotate; a unique constraint on the client UUID gives you sync idempotency for free – a duplicate insert simply fails and is ignored. Costs in DecimalField.

9. **Admin Dashboard** Dish, recipe and ingredient master with costs · menu calendar · service-level and safety-margin configuration · forecast model settings · hall and device management · data-quality monitor showing missing actuals by service.

10. **Reporting & Analytics** Forecast against actual accuracy (MAPE) by meal and hall · waste quantity and cost by dish, meal, hall and period · top-ten waste dishes · cost-saving trend against baseline · attendance patterns by weekday, event and exam period · ingredient consumption against indent.

11. **Notifications & Communication** Daily preparation guidance to kitchen staff · missing-actuals reminder · high-waste alert to the mess manager · menu published to students.

12. **Third-Party Integrations** Access-control or RFID system for attendance, ERP for student registration and mess fee category, procurement system export, email and SMS.

13. **Expected Deliverables** Per CES, plus a baseline waste measurement covering at least 4 weeks pre-deployment, so the saving can be evidenced.

14. **Development Milestones** M0 discovery and recipe data capture (2w) · M1 masters, attendance, calendar (3w) · M2 analysis and forecasting (2w) · M3 planning and actuals (2w) · M4 waste costing and reporting (2w) · M5 UAT and production (1w).

15. **Testing** Per CES. Specific: offline attendance capture and sync, including a duplicate-submission case; forecast reproducibility; waste cost arithmetic verified against a manual calculation for a full week.

16. **Deployment**

TRACK J · Node.js + MongoDB. Per CES. The kitchen PWA is served with a cache-first service worker and versioned assets.

TRACK P · Python + Django + PostgreSQL. Per CES. Serve the PWA from Django with correct cache headers, or as static files behind the CDN; the service worker is plain JavaScript and is not a Django concern.

17. **Documentation** Per CES, plus the forecast method written out in plain language for the mess manager – someone who is not a programmer must be able to see why the number is what it is.

18. **Skills Students Need**

TRACK J · Node.js + MongoDB. TypeScript · Node.js · MongoDB time-series aggregation · offline-first PWA · time-series forecasting basics · tablet UX for industrial environments.

TRACK P · Python + Django + PostgreSQL. Python · pandas · Django ORM aggregation · Celery · service workers and IndexedDB · basic statistics · tablet UX for industrial environments.

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with time-series data and basic statistics; one willing to build an offline-capable PWA and test it in an actual kitchen. *Both tracks*: service workers and offline sync appear in no syllabus and are the real learning here.

20. **Estimated Complexity and Semester Fit** Medium. 500–650 hours. One semester for a team of 4 if forecasting is a statistical baseline; add a second semester if a learned model is wanted after a term of data.

21. Suggested Student Team 4 students – tech lead · backend and forecasting · frontend and PWA · QA, documentation and field testing.

22. Acceptance Criteria

- 3,000 attendance records captured in a 45-minute window across halls without loss
- Attendance capture works offline and syncs without duplication – proven by test
- 7-day forecast generated daily with a confidence range
- Ingredient indent generated from the forecast and exportable
- Waste quantity and cost reported per dish and per service, reconciling to a manual calculation • Forecast accuracy (MAPE) reported and improving over an 8-week measurement period • Kitchen screens usable on a tablet with gloved hands, verified in a usability test with actual staff

AI/ML potential: Genuinely warranted here once 6–12 months of data exists – gradient boosting on attendance with weather, event and menu features. Do not start with it.

Commercialisation: Strong for residential campuses, hospitals and corporate canteens; the waste-cost reporting is the sales argument.

P06 – Placement Skill Matcher and Readiness Analyser

1. Project Title Evidence-Based Placement Readiness and Skill-Gap Platform. *Track J:* Next.js + NestJS + MongoDB. *Track P:* Django + DRF + PostgreSQL.

2. Project Overview *Description.* Builds a verified skill profile for each student from academic results, project work, assessments, mock interviews and external evidence; models recruitment drives and their requirements; computes readiness and specific gaps per drive; converts gaps into a time-bound preparation plan; and gives the Placement Cell a batch-level readiness view before a drive opens. *Problem.* Placement preparation is driven by CGPA and generic training. A student can be formally eligible for a drive and still be unprepared, and nobody knows which students or which skills until the drive fails. *Target users.* Students, faculty, placement officers, placement head, recruiters (restricted), administrators. *Expected outcome.* Skill gaps identified weeks before a drive, targeted training instead of generic training, and a defensible batch readiness picture.

Critical design distinction: eligibility and readiness are separate. A student may be eligible and unready, or ready and ineligible. The data model must keep these as independent computations, and the interface must never conflate

them.

3. Project Objectives

- Maintain an institutional skill taxonomy with synonyms and levels
- Build student skill profiles from multiple evidence sources with verification status
- Model drives, roles and requirements
- Compute eligibility and readiness independently, with explanations
- Produce ranked, actionable gaps and a preparation plan
- Provide controlled recruiter access limited to registered students in an approved drive
- Provide batch and cohort readiness analytics to the Placement Cell

4. Key Features Skill taxonomy with levels and equivalences · multi-source evidence ingestion · verified and unverified evidence distinction · mock-interview and assessment capture · readiness scoring by dimension – technical, aptitude, communication, project depth · drive and eligibility management · per-drive gap analysis · preparation planner with schedule · recruiter portal with scoped visibility · batch readiness dashboards.

5. Functional Modules

M1 Skill Taxonomy Skills, levels, synonyms, clusters, role mappings	Yes – build first
M2 Evidence Ingestion Academic, project, certification, assessment and external evidence	Yes
M3 Evidence Verification Faculty verification workflow; verified and unverified flags	Depends on M2
M4 Student Skill Profile Aggregates evidence into levelled skill claims	Depends on M1, M2, M3
M5 Assessment & Mock Interview Rubric-based evaluation capture and history	Yes
M6 Readiness Engine Dimension-wise readiness with explanation	Depends on M4, M5
M7 Drive & Eligibility Management Drives, roles, criteria, registration, shortlists	Yes
M8 Match & Gap Analysis Profile against drive requirement comparison	Depends on M4, M7
M9 Preparation Planning Gap to time-bound plan with resources and checkpoints	Scoped recruiter search; institutional analytics Depends on M7
M10 Recruiter Portal & Batch Reporting	Depends on M8

6. User Roles & Permissions Student – own profile, readiness, gaps, evidence and preparation plan; cannot

self-verify evidence. Faculty – record mock-interview evaluations; verify project and skill claims for own students. Placement Officer – drives, criteria, shortlists, batch reports, student detail. Placement Head – all of the above plus recruiter access control and institutional analytics. Recruiter – only students registered for their own approved drive, only fields approved for release, rate-limited, fully logged, no bulk export. Administrator – skill framework, evidence rules, scoring configuration.

7. Technology Stack

TRACK J · Node.js + MongoDB. NestJS for the domain complexity. Meilisearch or Atlas Search for skill and candidate

search. The recruiter portal is a separate Next.js route group with its own session policy and stricter rate limits.

TRACK P · Python + Django + PostgreSQL. PostgreSQL full-text search (SearchVector, SearchRank) with pg_trgm for fuzzy skill names – reach for Meilisearch only if that proves insufficient. The recruiter portal is a separate Django app with its own URL namespace, its own DRF permission classes and its own throttle rates; DRF's built-in throttling does this well. Batch readiness runs as a Celery task. Path P2 is reasonable here because the student dashboard is genuinely interactive.

8. Technical Requirements (deltas) *Frontend (both)*: radar visualisation of readiness dimensions; gap list ordered by impact; preparation plan as a calendar; recruiter search with faceting. *Backend (both)*: readiness recomputation triggered by new evidence, debounced; batch readiness computation for a 5,000-student cohort as a background job; skill matching using taxonomy expansion, so a student with “React” matches a requirement for “frontend framework”. *Data model (both)*: Skill, SkillSynonym, SkillCluster, Student, EvidenceItem, Verification, Assessment, MockInterview, SkillProfile, ReadinessSnapshot, Drive, DriveRequirement, EligibilityRule, Registration, GapAnalysis, PreparationPlan, RecruiterAccessGrant, AccessLog. *API (both)*: /skills/*, /students/:id/profile, /students/:id/readiness, /drives/*, /drives/:id/match, /preparation-plans, /recruiter/*. *Security (both)*: recruiter access is the highest-risk surface in this project – time-boxed grants, per-drive scoping, field-level release control, full access logging, no export endpoint, and an explicit negative test proving a recruiter cannot reach an unregistered student.

TRACK J · Node.js + MongoDB. Taxonomy expansion at query time through the search index; guard clauses in a dedicated authorisation module rather than scattered through controllers.

TRACK P · Python + Django + PostgreSQL. The recruiter's get_queryset filters to the approved drive *before* any other logic runs, and the serializer exposes only released fields – two independent layers, either of which alone would be sufficient and neither of which you rely on alone. Access grants carry an expiry checked in the permission class, not in a template.

9. Admin Dashboard Skill taxonomy editor with merge and synonym tools · evidence source configuration and weights · readiness model settings · drive management · recruiter grant management with expiry · verification queue · access-log review.

10. Reporting & Analytics Batch readiness distribution · skill supply against drive demand – the single most valuable report in this project, because it tells the training team what to run next term · gap frequency across the cohort · verification coverage · drive conversion funnel from eligible to registered to shortlisted to selected · training-effectiveness analysis · department comparison.

11. Notifications & Communication New drive matching profile · eligibility met or missed with reason · verification requested and completed · preparation checkpoint due · shortlist result · low-readiness nudge to student and mentor.

12. Third-Party Integrations ERP/SIS for academics, LMS for assessments, coding-assessment platforms of the HackerRank or Codility class by API or import, certification verification (P12 can supply this), email and SMS, SSO.

13. Expected Deliverables Per CES, plus a populated skill taxonomy for at least three target roles – software development, data, testing.

14. Development Milestones M0 discovery and taxonomy workshop (2w) · M1 taxonomy, evidence, verification (4w) · M2 profile and assessments (3w) · M3 readiness and gap analysis (4w) · M4 drives and preparation planning (3w) · M5 recruiter portal and analytics (3w) · M6 UAT, security review, production (2w).

15. Testing Per CES. A dedicated authorisation test suite for the recruiter portal is a named deliverable. End-to-end: evidence added, verified, profile updates, drive published, gap analysis, preparation plan, recruiter views shortlist.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. The recruiter portal deployed as a separate route group with independent rate limiting at the edge.

TRACK P · Python + Django + PostgreSQL. Per CES. The recruiter portal may share the process but must not share the session cookie domain or the throttle scope; document both settings.

17. **Documentation** Per CES, plus a documented data-release policy stating exactly which student fields a recruiter may see.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · NestJS · MongoDB · search engines and taxonomy modelling · scoring and matching algorithms · data visualisation · strict RBAC and ABAC.

TRACK P · Python + Django + PostgreSQL. Python · Django and DRF permission classes and throttling · PostgreSQL full-text search · taxonomy modelling · scoring algorithms · Celery · INT252 for the dashboard.

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member strong on access control, because the recruiter portal is the highest-risk surface here; one comfortable with search and taxonomy modelling. *Track P gaps:* DRF, Postgres full-text search, Celery.

20. **Estimated Complexity and Semester Fit** Large. 1,200–1,600 hours. Two semesters for a team of 5, or three parallel module teams (M1–M3 taxonomy and evidence · M4–M6 profile and readiness · M7–M10 drives, gaps and recruiter portal).

21. **Suggested Student Team** 5 students – tech lead · 2 backend, one owning authorisation · frontend and visualisation · QA and documentation.

22. Acceptance Criteria

- Skill profile assembled from at least four evidence types with verification status visible
- Eligibility and readiness computed and displayed independently
- Gap analysis produces ranked, specific, actionable gaps for any student against any drive
- Preparation plan generated with dated checkpoints
- Recruiter cannot access any student outside their approved drive – proven by negative tests
- All recruiter access logged and reviewable
- Batch readiness for 5,000 students computed as a background job in under 15 minutes
- Skill supply-against-demand report produced for a real drive

AI/ML potential: Substantial and well-founded – resume and project text to skill extraction with a language model under human verification, selection-probability modelling once two or more years of drive outcomes exist, and preparation-resource recommendation. Keep the human verification step permanently.

Commercialisation: The highest of any project in this portfolio. Placement readiness is a universal pain across Indian engineering education and there is no dominant incumbent.

P07 – Health Services and Emergency Response

1. **Project Title** Campus Health Services Scheduling and Emergency Response Coordination Platform. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + Channels + PostgreSQL.

2. **Project Overview Description.** Two connected capabilities: routine health-service access – medical officer

availability, appointment booking, live OPD queue – and emergency coordination – incident raising with location, routing to the on-duty response team, escalation on non-acknowledgement, and measurement of response times from raise to acknowledge to arrive to resolve. *Problem.* Students cannot see availability or queue status and turn up blind. Emergencies are reported by phone with no record, no routing guarantee and no measurement – so response performance is unknown and cannot be improved. *Target users.* Students, staff, medical officers, nursing staff, emergency responders, health-centre administrators, Dean of Student Welfare. *Expected outcome.* Shorter and measurable emergency response times, less crowding at the health centre, and a defensible incident record.

Scope constraint, to be stated in the project brief: this is a coordination platform, not a clinical system. It does not diagnose, does not recommend treatment, and does not constitute an electronic medical record. Any team proposing symptom-checking or triage AI has misread the brief.

3. Project Objectives

- Publish medical officer availability and manage appointment slots
- Operate a live OPD queue with visible position and estimated wait
- Capture emergency incidents with location and category
- Route incidents to the correct on-duty responder with acknowledgement tracking and automatic escalation
- Measure and report the full response timeline
- Maintain minimal, encrypted, access-controlled visit records

4. Key Features Officer roster and availability · appointment booking with cancellation and no-show handling · live queue with token display · emergency raise from mobile with location capture · on-duty responder roster · incident routing and escalation ladder · timeline stamping at each stage · response-time analytics · basic visit record · medicine stock levels (optional module).

5. Functional Modules

M1 Staff, Roster & Availability Medical officers, shifts, on-duty responders, leave Yes – build first	booking, rescheduling, cancellation, no-show M3 Live OPD Queue Token issue, queue position, estimated wait, display screen Depends on M1 Depends on M1
M2 Appointment Scheduling Slot generation, M4 Emergency Incident Capture	Raise incident, category, landmark, attachments location by GPS and campus Yes
M5 Routing & Escalation Assign to on-duty team, acknowledgement SLA, escalation ladder Depends on M1, M4	M6 Response Timeline Tracking Timestamped stage transitions Depends on M5
M7 Visit Record Minimal encrypted record of the encounter Response times, incident patterns, service load stock and reorder alerts Yes	Depends on M2 M8 Analytics & Reporting Depends on M6 M9 Inventory (optional) Medicine

6. User Roles & Permissions Student and Staff – book appointments, view queue, raise emergency, view own visit history. Medical Officer – own schedule, queue, record encounters. Nursing Staff – queue management, token issue. Emergency Responder – incident inbox, acknowledge, update status. Health Centre

Administrator – rosters, slots, categories, escalation rules. Dean of Student Welfare – analytics only, no individual health data. Administrator – configuration, no clinical data.

7. Technology Stack

TRACK J · Node.js + MongoDB. Socket.IO for live queue and incident status, Web Push or FCM for notifications, and a map component for campus location. Escalation timers as persisted BullMQ delayed jobs.

TRACK P · Python + Django + PostgreSQL. Django Channels over ASGI – Daphne or Uvicorn – for the live queue and incident status. This changes how the project deploys, so decide in week 2 and write it in the design document. Push through a Web Push library; the map is Leaflet in a template.

On both tracks the emergency path must have an SMS fallback. Do not build a life-safety-adjacent flow that depends solely on data connectivity.

8. **Technical Requirements (deltas)** *Frontend (both):* emergency raise reachable in two taps from app open, working on a poor connection and on low-end Android; the queue display screen as a separate kiosk route. *Backend (both):* incident routing within 5 s; escalation timers reliable under restart – persist them, do not hold them in memory; high-availability priority for the emergency path. *Data model (both):* Staff, Roster, AvailabilitySlot, Ap pointment, QueueToken, Incident, IncidentAssignment, IncidentTimeline, EscalationRule, VisitRecord (encrypted), Location, InventoryItem. *API (both):* /appointments, /queue, /incidents, /incidents/:id/acknowledge, /incidents/:id/status, /analytics/response-times. *Security (both):* health data is sensitive personal data under DPDP – field-level encryption, strict need-to-know access, a separate audit log for every health-record read, an explicit consent record, and configurable retention with deletion. *Availability (both):* the emergency module must degrade gracefully – if analytics or appointments are down, incident raising still works.

TRACK J · Node.js + MongoDB. Delayed BullMQ jobs survive a restart because they live in Redis; still assert this with a test that restarts the worker mid-incident.

TRACK P · Python + Django + PostgreSQL. Do not schedule escalation with a Celery countdown task – it is lost if the broker restarts. Store an `escalate_at` timestamp on the incident and run a Celery Beat sweeper every 30 seconds that picks up anything overdue. That design survives a restart; the naive one does not, and the acceptance criteria test exactly this. Encrypt visit fields with `django-cryptography` or `pgcrypto`.

9. **Admin Dashboard** Roster and on-duty management · escalation ladder configuration · incident category management · slot templates · live incident board · response SLA monitor · inventory alerts.

10. **Reporting & Analytics** Median and p90 response time by stage, category and location · incident heat map by campus zone and time of day · appointment utilisation and no-show rate · queue wait-time distribution · seasonal illness patterns, aggregate only · SLA breach report.

11. **Notifications & Communication** Appointment confirmation and reminder · queue position approaching · emergency: push, SMS and in-app to the on-duty team, with escalation to the next tier on non-acknowledgement.

12. **Third-Party Integrations** SMS gateway (mandatory), push service, campus map and geofencing, ERP for identity, ambulance and security contact directory, SSO.

13. **Expected Deliverables** Per CES, plus a documented escalation policy and a tested SMS fallback.

14. **Development Milestones** M0 discovery and clinical-workflow shadowing (2w) · M1 roster and appointments (2w) · M2 live queue (2w) · M3 emergency path (3w) · M4 records and analytics (2w) · M5 UAT with a live drill, production (2w).

15. **Testing** Per CES. A live emergency drill is a named acceptance activity, not a test-suite item. Specific tests: escalation fires correctly after a service restart; SMS fallback triggers when push fails; health-record access is denied to non-clinical roles.

16. **Deployment**

TRACK J · Node.js + MongoDB. Per CES. The realtime server and the API scale independently; sticky sessions or a Redis adapter for Socket.IO across replicas.

TRACK P · Python + Django + PostgreSQL. ASGI, not WSGI – Daphne or Uvicorn with a Redis channel layer, with Gunicorn workers retained for plain HTTP if preferred. Document the split. Celery Beat runs as exactly one instance.

17. Documentation Per CES, plus a data-protection impact note for the health data.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Node.js · WebSockets · push and SMS integration · geolocation · encrypted-field modelling · high-availability design · mobile-first PWA.

TRACK P · Python + Django + PostgreSQL. Python · Django Channels and ASGI · Celery Beat · push and SMS integration · geolocation · field-level encryption · CSE326 for the mobile-first PWA.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with real-time systems; the emergency path needs someone who will think about what happens when the network fails. *Track P*: this project has the longest gap list of any Medium project here – Channels, ASGI deployment, Celery, field encryption and geolocation are all outside the four courses. Choose it knowing that.

20. Estimated Complexity and Semester Fit Medium, upper end. 650–850 hours. Two semesters for a team of 4, or one semester for a team of 5 with the inventory module (M9) deferred.

21. Suggested Student Team 5 students – tech lead · 2 backend, one owning the emergency path · frontend and mobile PWA · QA and documentation.

22. Acceptance Criteria

- Emergency raised and routed to the on-duty responder within 5 seconds
- Escalation fires on non-acknowledgement, and survives an application restart
- SMS fallback verified in a live drill
- Emergency raise reachable in two taps and functional on 3G on a low-end device
- Full response timeline recorded and reportable
- Health records inaccessible to non-clinical roles – proven by negative tests
- Every health-record read logged separately and reviewably
- Live queue updates within 2 s across clients

AI/ML potential: Deliberately limited. Demand forecasting for staffing and incident-hotspot identification are appropriate. Symptom triage is out of scope and should be refused.

Commercialisation: Moderate; the emergency-coordination module is the differentiated part and generalises to any large campus or industrial site.

1. **Project Title** Institutional Research Expertise, Collaboration and Facility Management Platform. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. **Project Overview** *Description.* A consolidated register of researchers, expertise, publications, projects, funding calls and research facilities, with topic-based discovery across departments, student research-opportunity matching, collaboration suggestions, project milestone tracking and facility booking. *Problem.* Research capability is invisible across departmental boundaries. Faculty cannot find collaborators, students cannot find openings, equipment sits under-used, funding calls are missed, and institutional research reporting is assembled manually each year. *Target users.* Faculty researchers, research students, facility custodians, the Research Office, Dean of Research. *Expected outcome.* More cross-department collaboration, better facility utilisation, fewer missed funding deadlines, and research reporting that assembles itself.

3. Project Objectives

- Maintain verified researcher profiles with structured expertise
 - Maintain a publication register with author attribution and external-source import •
- Enable topic-based expertise discovery across the institution
- Publish research openings and match student applicants
 - Suggest collaboration opportunities based on expertise adjacency and co-authorship gaps •
 - Track projects, milestones, deliverables and deadlines
 - Manage facility discovery, booking and utilisation
 - Produce institutional research output reporting

4. **Key Features** Researcher profile with ORCID and Scopus import · expertise taxonomy · publication register with deduplication · full-text expertise search · research opening board and application workflow · collaboration suggestion engine · funding call register with deadline alerts · project and milestone tracking with at-risk flagging · facility catalogue, calendar and booking approval · institutional research dashboards.

5. Functional Modules

M1 Researcher & Expertise Profiles	first
Profiles, expertise tagging, verification	Yes – build
M2 Publication Register	Records, authors, deduplication, external import
M3 Expertise Discovery & Search	Yes
Topic search across profiles and publications	Depends on M1, M2
M4 Research Opportunities	Openings, requirements, applications, selection
M5 Collaboration Identification	Depends on M1
Expertise adjacency and co-authorship-gap	suggestions Depends on M1, M2
M6 Funding Call Register	Calls, eligibility, deadlines, alerts
M7 Project & Milestone Management	Yes – the most independently
M8 Facility & Asset Management	valuable module
M9 Reporting & Research Analytics	Output, collaboration, utilisation, at-risk reporting
Projects, milestones, dependencies, at-risk detection	Depends on M2, M7, M8
Yes	
Catalogue, availability, booking, approval, utilisation	

6. **User Roles & Permissions** Faculty Researcher – own profile, projects, openings, bookings, collaboration interests. Research Student – browse openings, apply, participate in assigned projects. Facility Custodian – facility records, approve bookings, log utilisation. Research Office – verify profiles, maintain funding calls, run

reports. Dean of Research – institutional analytics, at-risk projects. Administrator – expertise taxonomy, matching configuration.

7. Technology Stack

TRACK J · Node.js + MongoDB. Meilisearch or Atlas Search for expertise discovery, a calendar component for facilities, and connectors for ORCID, Crossref and Scopus. Embeddings-based expertise similarity is a v2 feature, not v1.

TRACK P · Python + Django + PostgreSQL. Python suits this project well. Import with habanero for Crossref and the ORCID API, inside Celery tasks. Deduplicate with rapidfuzz plus PostgreSQL pg_trgm trigram similarity – you can ask the database directly for near-matching titles, which is exactly the query this problem needs. Search with SearchVector across profile and publication text. Facility booking uses the ExclusionConstraint technique described in P20.

8. Technical Requirements (*deltas*) *Frontend (both)*: faceted expertise search; collaboration graph visualisation; facility booking calendar with conflict prevention; project milestone or Gantt view. *Backend (both)*: publication import with fuzzy deduplication and author disambiguation – the hardest part of this project, and it should be flagged in the posting; booking conflict prevention under concurrency; milestone at-risk computation as a scheduled job. *Data model (both)*: Researcher, ExpertiseTag, Publication, PublicationAuthor, ResearchProject, Milestone, ResearchOpening, Application, FundingCall, Facility, FacilitySlot, Booking, CollaborationSuggestion, AuditEntry. *API (both)*: /researchers, /publications, /search/expertise, /openings, /projects, /facilities, /bookings, /funding-calls, /analytics/research.

TRACK J · Node.js + MongoDB. Author disambiguation implemented in the service layer with a reviewable match queue; imports resilient to malformed external records.

TRACK P · Python + Django + PostgreSQL. Enable pg_trgm in a migration, add a GIN index on the normalised title, and let the database do the candidate generation before rapidfuzz scores the shortlist. Every merge decision is recorded so it can be reversed.

9. Admin Dashboard Profile verification queue · publication deduplication review · expertise taxonomy management · funding call curation · facility and custodian management · booking conflict monitor · at-risk project board.

10. Reporting & Analytics Publications by department, year, venue and type · cross-department collaboration rate · facility utilisation and idle capacity · project milestone adherence and at-risk list · funding application success rate · researcher productivity views, with an explicit caution: these are easily misused and access should be restricted to the Dean of Research.

11. Notifications & Communication Funding deadline countdown · milestone due and overdue · booking approved or rejected · new opening matching interests · collaboration suggestion · verification requested.

12. Third-Party Integrations ORCID, Crossref, Scopus or Web of Science (licence-dependent), Google Scholar (import only, no scraping), institutional repository, Google or Outlook calendar for facility bookings, SSO.

13. Expected Deliverables Per CES, plus imported publication data for at least two departments as a seeded baseline.

14. Development Milestones M0 discovery and taxonomy (2w) · M1 profiles and publications with import (4w) · M2 search and collaboration (3w) · M3 facility management (3w) · M4 openings, funding, projects (4w) · M5 analytics, UAT, production (3w).

15. Testing Per CES. Specific: publication deduplication against a curated fixture set with known duplicates and name variants; concurrent booking attempts on the same slot; import resilience to malformed external records.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. The search service runs as its own container with a documented reindex procedure.

TRACK P · Python + Django + PostgreSQL. Per CES. No search service to operate if you stay on PostgreSQL – one fewer moving part, and the reason to try Postgres search first.

17. Documentation Per CES, plus a written note on the deduplication rules and their known failure cases.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Node.js · MongoDB · search engines · fuzzy matching and entity resolution · calendar logic · graph visualisation with D3 or Cytoscape · third-party API integration.

TRACK P · Python + Django + PostgreSQL. Python · rapidfuzz and entity resolution · PostgreSQL pg_trgm and full-text search · Celery · third-party API clients · range types and exclusion constraints · graph visualisation in the browser.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. The publication-import module needs someone patient with messy real-world data. Prior exposure to information retrieval helps on either track.

20. Estimated Complexity and Semester Fit Large. 1,000–1,400 hours. Best allocated as three independent projects: M8 facility and asset management (Medium, one semester, genuinely useful on its own) · M1+M2+M3 profiles, publications and discovery (Medium to Large) · M4+M6+M7 openings, funding and projects (Medium).

21. Suggested Student Team 4–5 students per module team – tech lead · 2 backend · frontend · QA and documentation.

22. Acceptance Criteria

- Publications imported from at least two external sources with deduplication accuracy $\geq 95\%$ on the fixture set
- Expertise search returns relevant researchers across departments in under 1 s
- Facility double-booking impossible under concurrent requests – proven by test
- Project milestones flagged at-risk automatically with configurable thresholds
- Funding deadline alerts fire at configured intervals
- Institutional research report generated for a full academic year and exportable

Modules worth allocating as separate projects: M8 (Facility & Asset Management) is a viable standalone Small or Medium project and delivers value on its own. If capacity is constrained, allocate M8 first and defer the rest.

AI/ML potential: Embeddings-based expertise similarity and collaboration recommendation; automatic expertise extraction from publication abstracts.

Commercialisation: Moderate – competes with established research information management systems (Pure, Converis), so target the mid-market Indian private university where those are unaffordable.

P09 – Lost and Found Management System

1. **Project Title** Campus Lost and Found Platform with Ownership Verification and Custody Chain. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. **Project Overview** *Description.* One place to report lost and found items, with photo and description matching, deliberately withheld identifying details used later for ownership verification, and a supervised handover recorded as a custody chain. *Problem.* Lost property is handled by notice boards and WhatsApp groups. There is no matching, no ownership verification, and no record of what was handed to whom. *Target users.* Students, staff, security personnel, campus administration. *Expected outcome.* Higher return rate, verified handovers, and an auditable custody record.

Design idea worth preserving: the reporter of a found item records identifying details that are never shown publicly. A claimant must answer verification questions drawn from those hidden details. This is the mechanism that makes the system trustworthy and it should be built as a first-class feature, not an afterthought.

3. Project Objectives

- Capture lost and found reports with photos, category, location and time
- Match found items to lost reports on description, image, location and time proximity •
- Hold identifying details privately for claimant verification
- Support a verification challenge before any handover
- Record supervised handover with custody chain and proof
- Manage unclaimed-item disposal after a configured period

4. **Key Features** Lost and found reporting with photo upload · category taxonomy and campus location picker · automated match suggestions · private identifying-detail capture · claim workflow with verification challenge · security-supervised handover with signature or OTP · custody chain record · unclaimed-item ageing and disposal workflow · public item board with redacted details · search and filters.

5. Functional Modules

M1 Item Reporting	Lost and found report capture with media	Yes
M2 Category & Location Taxonomy	tree	Yes
Item categories, attributes, campus location		
		time-proximity matching; optional image similarity
M3 Matching Engine	Text, attribute, location and	Depends on M1, M2
		Hidden fields; challenge
		scoring
M4 Private Detail & Verification	generation; claimant response	Depends on M1

	supervised handover, OTP or signature, proof photo
M5 Claim Workflow Claim submission, verification, approval or rejection, competing claims	Depends on M4 Depends on M5
M6 Custody & Handover Item custody location,	
M7 Disposal & Ageing Unclaimed ageing, escalation, disposal record	Depends on M6 M8 Public Board
& Search Redacted public listing with filters	Depends on M1

6. User Roles & Permissions Student and Staff – report items, browse the redacted board, submit claims, view own reports and claims. Security Personnel – custody management, verification review, supervised handover, disposal. Campus Administrator – categories, locations, retention periods, reports. System Administrator – matching configuration, user management.

7. Technology Stack

TRACK J · Node.js + MongoDB. Sharp for image resizing and automatic EXIF stripping. Text matching with MongoDB text search or Meilisearch is sufficient for v1. Perceptual hashing or a lightweight embedding model for image similarity is a stretch goal only.

TRACK P · Python + Django + PostgreSQL. Pillow for resizing and EXIF stripping – a photo can carry the exact GPS coordinates of whoever took it, so strip before storing, not after. Matching with PostgreSQL full-text search plus pg_trgm. Path P3 is worth considering if the public board should be indexed by search engines.

Do not let a team gold-plate this with a vision model in the first release. Text and attribute matching is what the acceptance criteria measure.

8. Technical Requirements (deltas) *Frontend (both)*: mobile-first reporting flow completable in under 60 seconds; camera capture; location picker on a campus map; public board with infinite scroll. *Backend (both)*: match suggestions generated on report creation and refreshed nightly; private fields stored encrypted and never included in any public response – enforce this structurally, not by remembering to omit them. *Data model (both)*: Item, ItemCategory, Location, PrivateDetail (encrypted), MatchSuggestion, Claim, VerificationChallenge, CustodyRecord, Handover, DisposalRecord, AuditEntry. *API (both)*: /items, /items/:id/matches, /claims, /claims/:id/verify, /handovers, /board (public, redacted). *Security (both)*: the public board endpoint must be covered by an explicit test asserting that no private field ever appears in its response, for every item state.

TRACK J · Node.js + MongoDB. Enforce redaction in a serialisation layer – a dedicated public DTO that the private fields cannot reach.

TRACK P · Python + Django + PostgreSQL. Keep private details in a separate model with its own manager, and give the public board its own serializer or template context that structurally cannot include them. A related model with restricted access is harder to leak by accident than a field you must remember to exclude.

9. Admin Dashboard Item inventory by custody location · pending claims and verification queue · match review · ageing and disposal queue · category and retention configuration · return-rate reporting.

10. Reporting & Analytics Items reported against returned, giving the return rate · median time to return · loss hotspots by campus location and time · category distribution · unclaimed volume and disposal · verification failure rate, which is a signal for false claims.

11. Notifications & Communication Possible match found for your lost item · claim submitted on your found item · verification required · handover scheduled · unclaimed item ageing warning.

12. Third-Party Integrations ERP and SSO for identity, object storage for images, email and SMS, optionally campus CCTV location reference – metadata only, no video.

13. Expected Deliverables Per CES.

14. Development Milestones M0 discovery (1w) · M1 reporting and public board (2w) · M2 matching and private verification (2w) · M3 claims and handover (2w) · M4 disposal, reports, UAT, production (1w).

15. Testing Per CES. Specific: private fields never leak on any public endpoint, tested exhaustively across item states; competing claims on one item; verification challenge scoring; handover requires the security role.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. The public board benefits from static generation or edge caching.

TRACK P · Python + Django + PostgreSQL. Per CES. Cache the public board with Django's cache framework, or serve it from Next.js on Path P3 if search indexing matters.

17. Documentation Per CES, plus a written statement of exactly which fields are public and which are private, in language the security office can check.

18. Skills Students Need

School of Computer Science & Engineering Page 34 of 75
EduRev Project List · Dual Stack · P01–P20

TRACK J · Node.js + MongoDB. TypeScript · React/Next.js · Node.js · MongoDB text search · image handling · mobile-first UX.

TRACK P · Python + Django + PostgreSQL. Python · Django models, forms and permissions · Pillow · PostgreSQL full-text search · CSE326 for the mobile-first flow.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. No specialist prerequisites – this is the recommended first project for a team that has not shipped before, on either track. *Track P*: everything except image handling sits inside INT253 and CSE326.

20. Estimated Complexity and Semester Fit Small. 200–300 hours. One semester for a team of 2–3, comfortably.

21. Suggested Student Team 3 students – tech lead · full-stack · frontend, QA and documentation.

22. Acceptance Criteria

- Report submission completes in under 60 seconds on mobile
- Match suggestions returned for at least 70% of seeded matching pairs in the test fixture
- Private details never appear in any public response – proven exhaustively
- Claim requires passing the verification challenge before handover is permitted
- Handover recorded with custody chain and proof
- Unclaimed items age into the disposal queue at the configured interval

Recommended as a semester-1 allocation. Unambiguous scope, real users, one interesting privacy requirement, and comfortably deliverable by three students in fourteen weeks.

Commercialisation: Low as a standalone product; high as a bundled module in a campus operations

P10 – Campus Issue Management and Network Quality Analysis

1. **Project Title** Campus Grievance, Maintenance Ticketing and Network Quality Analytics Platform. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. **Project Overview** *Description.* A unified ticketing system for campus issues – maintenance, facilities, IT, network, hostel, academic administration – with categorisation, SLA-driven routing, escalation and resolution tracking, combined with a network quality module that aggregates connectivity complaints and measurement data to identify weak coverage zones. *Problem.* Complaints are raised through disconnected channels – verbal, WhatsApp, email, physical registers. Nothing is measured, ownership is unclear, escalation is informal, and recurring problems are never identified because there is no data. *Target users.* Students, staff, maintenance and IT teams, department heads, facility managers, campus administration. *Expected outcome.* Every issue tracked with an owner and an SLA, recurring problems visible, and network weak spots identified with evidence rather than anecdote.

3. Project Objectives

- Provide one intake channel for all campus issues with structured categorisation
- Route automatically to the responsible team based on category and location
- Enforce SLA with escalation and reminders
- Track the full lifecycle with status visibility for the reporter
- Aggregate network-quality complaints and client-side measurements into coverage analytics
- Report recurring issues, team performance and asset failure patterns

4. **Key Features** Multi-channel intake – web, mobile, QR code on physical assets · category and location taxonomy · auto-routing rules · SLA timers with escalation ladder · duplicate detection and merging · work-order assignment · resolution with proof photo · reporter feedback and reopen · network speed and latency test capture · coverage heat map · recurring-issue detection · team performance dashboards.

5. Functional Modules

M1 Issue Intake & Categorisation	Reporting across channels; category, location, media, QR reassignment	asset linking Yes – build first
M2 Routing & Assignment Rule-based routing by category, location and severity; manual	Depends on M1	
M3 SLA & Escalation Timers, reminders, escalation ladder, breach recording	Depends on M2	
M4 Work Order & Resolution	proof Depends on M2	Similarity on text, location
M5 Duplicate Detection & Merge Assignment, status, parts and effort, resolution	and time Depends on M1	
M6 Feedback & Reopen Reporter satisfaction, reopen within a window	Depends on M4	
M7 Network Quality Capture	Client-side speed and latency test, location tagging, complaint	linkage Yes – fully independent
M8 Coverage Analytics Aggregation into zone-level quality; weak-spot identification	Depends on M7	
M9 Dashboards & Reporting	Volume, SLA, team performance, recurring issues, asset failure	Depends on M3, M4

6. User Roles & Permissions Student and Staff – raise issues, track own tickets, provide feedback, run the network test. Technician or Field Staff – assigned work orders, status updates, resolution proof. Team Supervisor – team queue, assignment, SLA monitoring. Facility or IT Manager – cross-team analytics, routing rules, escalation configuration. Department Head – issues affecting own department. Campus Administration – institution-wide analytics. Administrator – taxonomy, SLA and routing configuration.

7. Technology Stack

TRACK J · Node.js + MongoDB. BullMQ for SLA timers, persisted and restart-safe; Socket.IO for live queue updates; Leaflet or Mapbox GL for coverage visualisation; QR generation for asset tagging. Network measurement runs client-side in the browser – no native app for v1.

TRACK P · Python + Django + PostgreSQL. SLA deadlines follow the P07 sweeper pattern: store breach_at on the ticket and sweep with Celery Beat rather than scheduling one task per ticket – with 500 tickets an hour, per-ticket tasks are the wrong shape. QR codes with the qrcode library; Leaflet in a template. Use PostGIS only if you genuinely need spatial queries – for campus zones a foreign key to a Zone model is simpler and enough.

8. Technical Requirements (deltas) *Frontend (both)*: issue reporting in under 45 seconds on mobile with a photo; QR scan pre-fills asset and location; technician view optimised for one-handed field use; heat map with zone drill-down. *Backend (both)*: SLA timers must survive restarts; routing rules configurable without deployment; duplicate detection on submission with a merge suggestion. *Data model (both)*: Issue, IssueCategory, Location, Asset, RoutingRule, Assignment, SlaPolicy, SlaEvent, WorkOrder, Resolution, Feedback, NetworkTest, CoverageZone, Team, AuditEntry. *API (both)*: /issues, /issues/:id/assign, /issues/:id/resolve, /work-orders, /network-tests, /analytics/coverage, /admin/routing-rules. *Performance (both)*: must absorb a burst of 500 issues in an hour – a hostel water outage generates exactly this – without queue degradation, and auto-group them as related.

TRACK J · Node.js + MongoDB. Duplicate detection over the text index; the burst test asserts grouping as well as throughput.

TRACK P · Python + Django + PostgreSQL. Duplicate detection with pg_trgm similarity over title and description,

bounded by location and a time window – one indexed query, not a scan. Bulk-create the burst rather than looping saves.

9. **Admin Dashboard** Category and SLA policy configuration · routing rule builder with a test harness · team and technician management · asset registry with QR generation · live issue board · SLA breach monitor · duplicate merge queue.

10. **Reporting & Analytics** Volume by category, location, building and time · SLA compliance by team and category · median and p90 resolution time · reopen rate · recurring-issue and repeat-asset-failure detection · technician workload and throughput · network coverage heat map with weak-zone ranking · complaint-to-measurement correlation · satisfaction trend.

11. **Notifications & Communication** Ticket acknowledged, assigned, resolved · SLA approaching and breached, to the supervisor · escalation · feedback request · mass notification when a known outage generates many tickets.

12. **Third-Party Integrations** ERP and SSO, email and SMS, asset management if one exists, network monitoring system for backbone data (optional), map tiles.

13. **Expected Deliverables** Per CES, plus a seeded category, SLA and routing configuration reflecting real campus teams.

14. **Development Milestones** M0 discovery and category workshop (2w) · M1 intake and routing (3w) · M2 SLA and work orders (3w) · M3 duplicates and feedback (2w) · M4 network module and coverage analytics (3w) · M5 dashboards, UAT, production (2w).

15. **Testing** Per CES. Specific: SLA timers survive restart; the escalation ladder fires in order; a burst of 500 issues is handled with correct grouping; routing rules evaluate correctly against a fixture set; network test results aggregate correctly by zone.

16. **Deployment**

TRACK J · Node.js + MongoDB. Per CES. Realtime and worker processes separate from the API.

TRACK P · Python + Django + PostgreSQL. Per CES. Celery worker and Beat containers alongside the web container; Channels only if live board updates are in scope.

17. **Documentation** Per CES, plus the routing and SLA configuration documented as data, so a facility manager can change it without the team.

18. **Skills Students Need**

TRACK J · Node.js + MongoDB. TypeScript · Node.js · MongoDB · BullMQ and Redis · WebSockets · geospatial data and mapping · mobile-first PWA · workflow and SLA engine design.

TRACK P · Python + Django + PostgreSQL. Python · Django · Celery Beat sweepers · pg_trgm similarity · Leaflet · QR generation · CSE326 for the mobile and technician views.

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with background job queues and time-based logic; one with mapping data for the network coverage module. *Track P gaps:* Celery, Leaflet, QR generation.

20. **Estimated Complexity and Semester Fit** Medium, upper end. 700–900 hours. One semester for a team of 5 covering the ticketing core (M1–M6, M9); allocate the network quality module (M7+M8) as a separate parallel project – it is fully independent and makes a good second team.

21. **Suggested Student Team** 5 students on the ticketing core · 3 students on the network quality module – each with a tech lead, backend, frontend and shared QA.

22. **Acceptance Criteria**

- Issue raised in under 45 seconds on mobile including a photo
- Auto-routing correct for the full fixture set of routing rules

- SLA timers and escalation survive an application restart
- 500-issue burst absorbed with related-issue grouping
- Network coverage heat map produced from real client measurements with zone ranking
- Recurring-issue detection identifies a seeded repeat failure
- SLA compliance and team performance reports exportable

AI/ML potential: Automatic categorisation from free-text descriptions; predictive maintenance from repeat asset failures; complaint-volume forecasting around known events.

Commercialisation: Strong. Campus ticketing is a universal need and the network-quality module is an unusual differentiator that no generic service-management tool offers.

P11 – Alumni Mentorship and Career Network

1. **Project Title** Verified Alumni Mentorship Matching and Engagement Platform. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. **Project Overview** *Description.* Verifies alumni identity, captures professional expertise, mentoring interests, capacity and availability, matches students to mentors against declared career goals, and supports the mentoring relationship through requests, scheduling, goals, action items and feedback – while keeping alumni contact details private and giving the institution engagement visibility. *Problem.* Alumni goodwill exists but is unstructured. Connections are ad hoc, unevenly distributed, invisible to the institution, and decay without follow-up. Alumni are also over-contacted by students who found them on LinkedIn. *Target users.* Students, alumni mentors, alumni relations officers, placement officers. *Expected outcome.* A sustained mentoring programme with measurable engagement, protected alumni capacity, and outcomes the institution can report.

Design constraint that determines success: protect the mentor's time. Capacity limits, request throttling and easy declining must be first-class features. Alumni mentoring programmes fail because mentors get overwhelmed and disengage, not because students are uninterested.

3. Project Objectives

- Verify alumni identity against institutional records before granting mentor status

- Capture structured expertise, mentoring areas, capacity and availability
- Capture student career goals and interests
- Match and rank mentors with explained relevance
- Manage requests, acceptance, scheduling and session records
- Track goals, action items and feedback across the relationship
- Detect declining engagement and prompt follow-up
- Report programme outcomes without exposing personal contact data

4. **Key Features** Alumni verification workflow · professional profile with expertise tagging · capacity and availability management with hard limits · student career profile · ranked matching with explanations · request and accept workflow with expiry · in-platform scheduling with calendar integration · session logging · goal and action-item tracking · bidirectional feedback · engagement decay detection · programme analytics · in-platform messaging with contact details hidden.

5. Functional Modules

M1 Alumni Verification & Profile	Identity verification against records, professional profile, limits, available periods, pause and resume	expertise Yes – build first
M2 Capacity & Availability Concurrent mentee	Depends on M1	
M3 Student Career Profile Goals, interests, target roles and industries	Yes	
M4 Matching Engine Ranked mentor suggestions with explained relevance and capacity filtering	expiry, decline with optional reason Depends on M1, M2, M3	
M5 Request & Acceptance Requests, throttling,	Depends on M4	
M6 Scheduling & Sessions Slot proposal, booking, reminders, session records	Depends on M5	
M7 Goals, Actions & Feedback	Goal setting, action items, completion, bidirectional feedback	Depends on M6
M8 Engagement Monitoring Inactivity detection, nudges, relationship health	Depends on M6, M7	M9
Messaging In-platform messaging with contact-detail masking	Depends on M5	M10
Participation, outcomes, mentor effectiveness	Depends on M7, M8	Programme Analytics

6. **User Roles & Permissions** Student – own profile, browse matches, send requests (throttled), manage own mentoring relationships. Alumnus or Mentor – own profile, capacity, availability, respond to requests, run sessions, provide feedback. Alumni Relations Officer – verification, engagement monitoring, mentor effectiveness, programme reports. Placement Officer – mentoring participation in relation to placement outcomes, read-only. Administrator – matching weights, throttle limits, engagement settings.

7. Technology Stack

TRACK J · Node.js + MongoDB. Calendar integration with Google Calendar and Outlook via OAuth, video-call link generation, and Socket.IO for messaging. Contact-detail masking is a serialisation-layer concern and must be implemented there.

TRACK P · Python + Django + PostgreSQL. Calendar integration with google-api-python-client and the Microsoft Graph API via OAuth. Messaging with Channels, or plain polling if you want to hold the scope down – polling is an acceptable v1 here and removes an entire deployment concern. Masking belongs in one serializer or one template filter used everywhere, never repeated per view.

8. **Technical Requirements (deltas)** *Frontend (both):* mentor discovery with facets and clear capacity indication; a mentor dashboard that makes declining easy and guilt-free; mobile-friendly session management for busy

professionals. *Backend (both)*: matching computed on demand with capacity filtering; request throttling per student per period; automatic request expiry; an engagement-decay job running weekly. *Data model (both)*: *Alumnus*, *VerificationRequest*, *ExpertiseTag*, *MentorCapacity*, *AvailabilityWindow*, *StudentProfile*, *CareerGoal*, *MatchScore*, *MentorshipRequest*, *Mentorship*, *Session*, *Goal*, *ActionItem*, *Feedback*, *Message*, *EngagementSnapshot*. *API (both)*: */alumni/verify*, */mentors/search*, */requests*, */mentorships/:id/sessions*, */mentorships/:id/goals*, */messages*, */analytics/programme*. *Privacy (both)*: alumni email and phone are never exposed to students at any point, in any response, including matching results – test this explicitly.

TRACK J · Node.js + MongoDB. Capacity enforcement under concurrent acceptance handled with a conditional update that fails when the limit is already reached.

TRACK P · Python + Django + PostgreSQL. Capacity must hold when two students accept in the same instant: wrap acceptance in `transaction.atomic()` with `select_for_update()` on the mentor row, or add a `CheckConstraint` on the count. Expiry is a Celery Beat sweep over an `expires_at` column.

9. **Admin Dashboard** Verification queue with document review · mentor capacity overview and overload warnings · matching weight configuration · throttle and expiry settings · engagement health board · mentor effectiveness ranking · programme reporting.

10. **Reporting & Analytics** Active mentorships and participation rate · request acceptance rate · session frequency and completion · goal completion rate · mentor utilisation against capacity · engagement decay and recovery · mentoring participation correlated with placement outcomes · mentor and student satisfaction scores.

11. **Notifications & Communication** Verification status · new request, with easy decline · request accepted, declined or expired · session scheduled and reminder · action item due · inactivity nudge · feedback request.

12. **Third-Party Integrations** ERP or alumni records for verification, LinkedIn (optional, profile prefill only), Google and Outlook calendar, video-conferencing, email and SMS, SSO.

13. **Expected Deliverables** Per CES, plus a verified seed set of at least 50 alumni profiles.

14. **Development Milestones** M0 discovery (1w) · M1 verification and profiles (3w) · M2 matching and requests (3w) · M3 scheduling and messaging (3w) · M4 goals, feedback, engagement (2w) · M5 analytics, UAT, production (2w).

15. **Testing** Per CES. Specific: alumni contact data never appears in any student-accessible response; capacity limits cannot be exceeded under concurrent requests; request throttling enforced; the expiry job is correct.

16. **Deployment**

TRACK J · Node.js + MongoDB. Per CES. Messaging server separate if Socket.IO is used at scale.

TRACK P · Python + Django + PostgreSQL. Per CES. Polling-based messaging keeps this a plain WSGI deployment; choosing Channels adds ASGI, so make that choice deliberately.

17. **Documentation** Per CES, plus a written privacy statement listing every field visible to a student, which the alumni office can check without reading code.

18. **Skills Students Need**

TRACK J · Node.js + MongoDB. TypeScript · Next.js · Node.js · MongoDB · matching and ranking algorithms · OAuth and calendar APIs · real-time messaging · privacy-conscious API design.

TRACK P · Python + Django + PostgreSQL. Python · Django · OAuth against third-party calendars · row locking for capacity · Celery · ranking algorithms · privacy-conscious serializer design.

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with OAuth and third-party calendar APIs; the privacy requirement needs someone who will actually test that alumni contact details never leak. *Track P gaps*:

OAuth flows and, if chosen, Channels.

20. Estimated Complexity and Semester Fit Medium. 600–800 hours. Two semesters for a team of 4, or one semester for a team of 5 with in-platform messaging (M9) deferred to semester 2.

21. Suggested Student Team 5 students – tech lead · 2 backend, one owning matching and privacy · frontend · QA and documentation.

22. Acceptance Criteria

- Alumni verification workflow completes against institutional records with an audit trail
- Mentor capacity limits cannot be exceeded, including under concurrent acceptance
- Student request throttling enforced and configurable
- Alumni contact details never exposed – proven by exhaustive endpoint testing
- Matching returns ranked mentors with a stated reason for each match
- Sessions schedulable with calendar integration and reminders
- Engagement decay detected and nudges issued
- Programme report generated covering participation, sessions, goals and outcomes

AI/ML potential: Semantic matching between student goals and mentor experience using embeddings; predicting which mentorships will lapse, so intervention can be targeted.

Commercialisation: Moderate to strong; alumni engagement platforms are a real market, and the capacity protection design is a genuine differentiator against tools that simply expose a directory.

P12 – CertiScholar: Document Verification and Scholarship Recommendation

1. Project Title OCR-Based Document Verification and Explainable Scholarship Eligibility Platform. *Track J:* Next.js + Node.js + MongoDB plus a Python OCR service. *Track P:* Django + PostgreSQL, OCR in-process.

2. Project Overview *Description.* Takes a student document – marksheet, identity document, income certificate – through a defined pipeline: upload, OCR, classify, extract fields with confidence, validate by recomputation, verify against an authorised source, evaluate scholarship rules, and produce an explainable recommendation and report, with a full audit timeline. *Problem.* Scholarship and admission document handling is manual, repetitive, slow and inconsistent. Data is re-keyed, eligibility decisions are opaque, and there is no record of why a decision was reached. *Target users.* Students, verification officers, scholarship and student services officers, administrators. *Expected outcome.* Sharply reduced manual data entry, consistent and explainable eligibility decisions, and an auditable record for every application.

The constraint that must be carried into the build and into the project brief: OCR and internal consistency checks do not prove authenticity. Authenticity can only come from an authorised issuing authority or trusted credential source. The system must present its output as *extraction and consistency assessment*, never as *verified genuine*, unless an authorised verification provider has actually responded. Any team proposing AI fraud detection as a substitute has

misunderstood the requirement.

3. Project Objectives

- Accept documents securely with type, size and content validation
- Classify document type and route to the correct extraction profile
- Extract fields with per-field confidence, flagging low confidence for human review rather than guessing
- Recompute marks, totals and percentages and detect inconsistency with printed values • Verify against a configured authorised provider through a pluggable adapter
- Evaluate configurable scholarship rules with AND/OR logic
- Produce an explainable recommendation naming every condition evaluated
- Maintain a complete processing timeline and audit trail

4. Key Features Secure upload with malware scanning · OCR with provider abstraction · document classification · field extraction with confidence scores · human review and correction queue · academic recomputation and consistency checking · pluggable verification adapter with a clearly labelled simulated provider for non-production · rule-based scholarship engine with a no-code rule builder · explainable recommendation with condition-by-condition breakdown · processing timeline · downloadable reports · admin rule management.

5. Functional Modules

M1 Secure Document Intake Upload, validation, virus scan, storage outside the web root, pre-signed access	OCR: Tesseract, cloud vision or document AI Yes – build first Yes
M2 OCR & Text Extraction Provider-abstracted	
M3 Document Classification Identify document type, select extraction profile	Depends on M2
M4 Field Extraction & Confidence confidence; low-confidence flagging	percentages; compare with printed values
M5 Human Review & Correction Side-by-side document and field review; authorised correction with audit	Depends on M3 Depends on M4
M6 Academic Calculation & Consistency Per-field extraction with	Depends on M4
M7 Verification Adapter Pluggable authorised-source verification; simulated provider for demo	Yes – interface first

M8 Scholarship Rule Engine Configurable conditions with AND/OR logic, thresholds, categories Yes – the most reusable module	M9 Explainable Recommendation Condition-by-condition result and reasoning Depends on M6, M7, M8
M10 Timeline, Audit & Reporting	Stage timeline, actor log, dashboards, exportable reports Depends on all

6. User Roles & Permissions Student – upload own documents, view own processing status, extracted values, verification status and recommendation. Verification Officer – review queue, correct extracted fields (audited), handle exceptions, mark manual verification. Scholarship Officer – review eligibility and recommendations, generate reports, process awards. Administrator – scholarship rules, extraction profiles, verification provider configuration, user access.

7. Technology Stack

TRACK J · Node.js + MongoDB. OCR abstracted behind an interface from day one with at least two implementations – self-hosted Tesseract and one cloud provider (Google Document AI, AWS Textract or Azure Document Intelligence). A Python microservice is acceptable and likely for the OCR and extraction layer; the rest stays in the Node and TypeScript stack, which means an extra service, an extra deployment and an extra contract to maintain. BullMQ is mandatory – OCR is slow and must never run inside a request.

TRACK P · Python + Django + PostgreSQL. This is the strongest project on Track P. Everything needed is a Python library: pytesseract with Tesseract, OpenCV for deskewing and cleanup, and the cloud SDKs behind the same interface. No separate microservice is required – the OCR code is another Django app running inside Celery tasks, with per-stage status on the document row so a single stage can be retried. Bounding boxes stored as JSONB. Path P2, because the review screen is the one place where React genuinely earns its keep.

8. Technical Requirements (*deltas*) *Frontend (both)*: a side-by-side viewer with the document image and extracted fields, each field highlighting its source region on the image – the bounding-box overlay is a specified requirement, not a nice-to-have, because it is what makes review fast; confidence indicated per field; correction inline. *Backend (both)*: asynchronous pipeline with per-stage status and retry; per-field confidence thresholds configurable; a rule engine evaluating nested AND/OR without code changes; idempotent reprocessing. *Data model (both)*: Document, DocumentType, ExtractionProfile, ExtractedField, ConfidenceScore, Correction, ConsistencyCheck, VerificationRequest, VerificationResult, Scholarship, ScholarshipRule, RuleCondition, Application, Recommendation, ProcessingTimeline, AuditEntry. *API (both)*: /documents, /documents/:id/status, /documents/:id/fields, /documents/:id/verify, /scholarships, /scholarships/rules, /applications/:id/recommendation, /reports. *Security (both)*: documents contain identity and income data – encryption at rest, pre-signed time-limited access only, no direct URLs, a separate audit log for every document view, configurable retention with deletion, and access restricted by role and case assignment. *Performance (both)*: 500 documents processed per hour; a single-document pipeline under 60 seconds for a standard marksheet.

TRACK J · Node.js + MongoDB. The service contract between Node and the Python OCR worker must be versioned and documented; a failure in the OCR service must leave the document in a retryable state, not a lost one.

TRACK P · Python + Django + PostgreSQL. Rules stored as JSONB and interpreted in Python – the same technique as P02, and the module is deliberately reusable across both projects. Run OCR workers on a separate Celery queue so a slow document never delays rule evaluation.

9. Admin Dashboard Scholarship and rule builder – no-code, with a test harness where a hypothetical applicant is entered and the evaluation shown · extraction profile management per document type · confidence threshold configuration · verification provider configuration · review queue monitoring · processing throughput and failure rates · audit viewer.

10. Reporting & Analytics Applications by status and scholarship · extraction accuracy and correction rate by field and document type, which drives extraction-profile improvement · consistency-failure rate · verification

success rate · recommendation outcomes · officer throughput · processing time by stage · exportable award lists.

11. Notifications & Communication Document received, processing complete, review required, additional document needed, verification outcome, recommendation issued, award decision.

12. Third-Party Integrations OCR or document-AI provider, authorised verification services (university records, examination boards, DigiLocker where available), object storage, malware scanning, email and SMS, SSO. Every integration behind an adapter with a documented fallback to manual verification.

13. Expected Deliverables Per CES, plus at least three extraction profiles built for real document formats, a populated rule set for at least two live scholarships, and an accuracy benchmark report on a labelled test set of 200 documents.

14. Development Milestones M0 discovery, document sampling, extraction profile design (2w) · M1 intake and OCR pipeline (3w) · M2 classification and extraction with confidence (4w) · M3 review interface and consistency checking (3w) · M4 verification adapter and rule engine (3w) · M5 recommendation, timeline, reporting (3w) · M6 accuracy benchmarking, UAT, production (2w).

15. Testing Per CES. A labelled test set of 200 real documents is a client-supplied prerequisite, and field-level

extraction accuracy against it is an acceptance criterion. Specific: low-confidence fields always route to review; corrections audited; the rule engine evaluates the full condition fixture set; the simulated verification provider is unmistakably labelled and cannot be enabled in production configuration.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES, plus the OCR service deployed and scaled separately, with its own health checks and its own failure handling in the calling service.

TRACK P · Python + Django + PostgreSQL. Per CES. One codebase, two process types – web and Celery worker. The OCR worker image carries the Tesseract binary and OpenCV; document it, because it is the one non-obvious thing in the Dockerfile.

17. Documentation Per CES, plus written documentation of what the system does and does not establish about a document, in language suitable for a scholarship committee.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Node.js · Python for the OCR service · OCR and document AI · MongoDB · queue-based pipeline design · rules engines · document-viewer UI with bounding boxes · security-conscious file handling · cross-service contracts.

TRACK P · Python + Django + PostgreSQL. Python · pytesseract and OpenCV · Celery pipelines · DRF · JSONB rule interpretation · INT252 for the review interface · security-conscious file handling.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. At least one member with Python and practical machine-learning or computer-vision experience – an OCR pipeline is not learnable from scratch inside a semester alongside everything else. Prior coursework in image processing is a strong signal. *Track P:* this project needs the most self-learning of any on that track – OCR, OpenCV, Celery and DRF – and gives the most back.

20. Estimated Complexity and Semester Fit Large. 1,100–1,500 hours. Two semesters for a team of 5. Semester 1: M1–M6 intake, OCR, extraction, review and consistency. Semester 2: M7–M10 verification adapter, rule engine, recommendation and reporting. The rule engine (M8) is independently buildable and can run as a parallel project.

21. Suggested Student Team 5 students – tech lead · backend · OCR and extraction (Python) · frontend with the document viewer · QA and documentation.

22. Acceptance Criteria

- Field-level extraction accuracy $\geq 90\%$ on the 200-document labelled test set for standard marksheet formats, reported per field

- Every field below the confidence threshold routes to human review; nothing is silently guessed
- Consistency check detects a printed percentage that disagrees with subject marks
- Scholarship rules created and modified entirely through the admin interface with no deployment
- Recommendation lists every condition evaluated with its result
- Simulated verification provider clearly labelled and blocked in production configuration
- 500 documents processed per hour
- Every document view logged and reviewable
- Complete processing timeline available for any application

AI/ML potential: Layout-aware extraction models of the LayoutLM class for non-standard formats; anomaly detection on document images as a *review-prioritisation signal only*, never as an authenticity verdict.

Commercialisation: Strong. Document verification and scholarship processing is a repeated pain across admissions, scholarships, hostels and placements, and the rule engine (M8) is independently reusable across all of them.

P13 – Faculty Workload, Course Allocation and Timetable Engine

1. **Project Title** Integrated Faculty Workload, Course Allocation and Automated Timetabling Platform. *Track J:* Next.js + NestJS + MongoDB plus a Python constraint-solver service. *Track P:* Django + PostgreSQL, solver in-process.

2. **Project Overview Description.** One system covering three coupled problems currently solved separately and manually: computing and balancing faculty workload against institutional norms; allocating courses and sections to faculty against expertise, preference and workload; and generating a conflict-free timetable across faculty, student sections, rooms and labs. *Problem.* At a university with thousands of faculty and tens of thousands of students, allocation and timetabling take weeks of senior faculty time each term, are done in spreadsheets, produce conflicts discovered after publication, and distribute workload inequitably in ways nobody can evidence or dispute. *Target users.* Faculty, HoDs, timetable coordinators, deans, room and lab custodians,

students. *Expected outcome.* Timetable generation reduced from weeks to hours, zero hard conflicts at publication, and workload distribution that is visible, comparable and defensible.

This is the hardest project in the portfolio. Say so in the posting, budget accordingly, and do not award it to a team that has not shipped a scheduling or constraint-optimisation system before.

3. Project Objectives

- Model workload norms – teaching, lab, administrative, research and mentoring load – with configurable weights
- Compute per-faculty workload and compare against norms and peers
- Allocate courses and sections to faculty against expertise, preference, continuity and workload balance
- Generate conflict-free timetables honouring hard constraints and optimising soft ones
- Support human review, manual override and re-optimisation without losing prior decisions
- Publish timetables to faculty, students and room displays, with change management

4. Key Features Workload norm configuration · faculty expertise and preference capture · course and section demand planning · allocation engine with balance objectives · room, lab and equipment inventory · timetable constraint solver · conflict detection and explanation · manual override with automatic re-validation · scenario comparison · publication and version control · mid-term change request workflow · substitution and leave handling · personal timetable views.

5. Functional Modules

M1 Faculty Master & Expertise	Faculty records, designations, expertise, availability constraints, leave	Rooms, capacity, features, labs, equipment, availability windows
M2 Workload Norm & Computation	Configurable norms, weighted load computation, comparison	Yes – build first Depends on M1
M3 Course & Section Demand Planning	Courses, sections, expected enrolment, contact-hour patterns (L-T-P)	Yes
M4 Room, Lab & Resource Inventory		Yes – reusable, shared with P20
M5 Preference Capture	Faculty course and slot preferences with deadlines	Depends on M1, M3
M6 Allocation Engine	Course-to-faculty assignment optimising expertise, preference, continuity, balance	Depends on M2, M3, M5
M7 Timetable Constraint Solver	Slot assignment across faculty, sections and rooms; hard and soft constraints	Depends on M4, M6
M8 Conflict Detection & Explanation	Identifies and explains every conflict in human terms	Depends on M7
M9 Override & Re-optimisation	Pin decisions, re-solve the remainder, preserve manual choices	Depends on M7

M10 Publication & Change Management	Versioned publication, change requests, substitutions,	notifications Depends on M9
M11 Views & Dashboards	Faculty, student, room	and department views; workload equity reporting Depends on M10

6. User Roles & Permissions Faculty – own workload, preferences, timetable, change requests, leave. HoD – department allocation, workload balance, approvals. Timetable Coordinator – solver runs, constraints, overrides, publication. Room or Lab Custodian – resource records and availability. Dean – cross-department workload

equity, utilisation. Student – own section timetable. Administrator – norms, constraint configuration, solver parameters.

7. Technology Stack

TRACK J · Node.js + MongoDB. The constraint solver runs as a separate service, in Python with OR-Tools CP-SAT or an equivalent proven solver. The Node and TypeScript application owns data, workflow and interface; the solver service receives a model and returns assignments. That means a second language, a second deployment and a versioned contract between them – budget for it.

TRACK P · Python + Django + PostgreSQL. Here Track P has a real structural advantage: OR Tools is a Python library. Use `ortools.sat.python.cp_model` directly from a Celery task – no second service, no cross-language contract, no separate deployment. Store the model input and the seed so any run is reproducible.

On both tracks: do not accept a plan to write a bespoke heuristic timetabler. This is a well-studied constraint-satisfaction problem and reinventing it is the classic way these projects fail.

8. Technical Requirements (deltas) *Frontend (both):* interactive timetable grid with drag-to-move and instant conflict feedback; scenario comparison view; workload equity visualisation; faculty preference form with deadline enforcement. *Backend (both):* solver invocation as a long-running job with progress reporting; models serialisable and stored so a run is reproducible; support for pinning assignments and re-solving the remainder; incremental re-solve after a mid-term change without republishing the whole timetable. *Constraints – hard, never violated:* no faculty in two places at once; no section double-booked; no room double-booked; room capacity at least the section size; lab sessions only in appropriately equipped labs; faculty unavailability windows respected; maximum consecutive teaching hours; statutory rest gaps. *Constraints – soft, optimised:* faculty slot preferences; minimise gaps in faculty and student days; distribute a course's sessions across the week; avoid first and last slots where possible; keep a faculty member's classes in nearby buildings; balance workload variance across a department. *Data model (both):* Faculty, ExpertiseArea, WorkloadNorm, WorkloadSnapshot, Course, Section, ContactHourPattern, Room, Lab, Equipment, AvailabilityWindow, Preference, Allocation, TimetableRun, TimetableAssignment, PinnedAssignment, Conflict, Publication, ChangeRequest, Substitution. *API (both):* /faculty/*, /workload/*, /allocations/solve, /timetables/solve, /timetables/:runId, /timetables/:id/publish, /change-requests. *Performance (both):* a full timetable for 800 sections, 400 faculty and 200 rooms produced within 30 minutes; incremental re-solve after a single change within 2 minutes.

9. Admin Dashboard Workload norm configuration · constraint editor with hard and soft classification and weights · solver run history with parameters and reproducibility seed · scenario comparison · conflict board · publication version control with rollback · resource utilisation monitor.

10. Reporting & Analytics Workload distribution and variance by department and designation, as equity reporting · preference satisfaction rate per faculty · room and lab utilisation with idle capacity · timetable quality metrics – gaps, compactness, spread · change-request volume and reasons · comparison against the previous term.

11. Notifications & Communication Preference window opening and closing · allocation published · timetable published and changed · change request status · substitution assigned · leave affecting scheduled sessions.

12. Third-Party Integrations ERP/SIS for faculty, courses and enrolment; P14 for actual section sizes after registration; P20 for shared room inventory; calendar export (iCal) for faculty and students; digital display boards; SSO.

13. Expected Deliverables Per CES, plus a documented constraint model, a solver tuning guide, and a full timetable generated for one real department as the acceptance artefact.

14. Development Milestones M0 discovery, constraint elicitation workshop, model specification (4w) · M1 M1+M3+M4 masters (4w) · M2 M2+M5 workload computation and preference capture (3w) · M3 M6 allocation engine (4w) · M4 M7+M8 timetable solver and conflict explanation (6w) · M5 M9+M10 override, re-optimisation, publication, change management (5w) · M6 M11 views, analytics, UAT with a real department, production (4w).

15. Testing Per CES. Hard-constraint satisfaction must be verified by an independent validator written separately from the solver – a program that reads a published timetable and asserts every hard constraint, so

the solver cannot mark its own homework. Load test: a full-scale solve. Regression: a fixture of known-difficult scheduling scenarios.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. Solver service scaled independently; solve jobs isolated so a long run never blocks the application.

TRACK P · Python + Django + PostgreSQL. Per CES. A dedicated Celery queue and worker for solve jobs, sized for CPU rather than concurrency. The solver is CPU-bound: give its worker its own container and do not let it share with short tasks.

17. Documentation Per CES, plus the constraint model specification and a “how to add a new constraint” guide, since constraints change every year and the client must not need the original team for that.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · NestJS · Python with OR-Tools or an equivalent CP/MIP solver · constraint modelling · MongoDB · complex interactive grid UI · long-running job orchestration · cross-service contracts.

TRACK P · Python + Django + PostgreSQL. Python · OR-Tools CP-SAT · constraint modelling · Celery for long-running jobs · Django ORM · CSE326 for the timetable grid.

19. Prerequisites and Readiness Returning students who have already completed one project in this programme; at least one member with a genuine optimisation background – operations research, constraint programming, or a strong algorithms track. Prior exposure to OR-Tools or an equivalent solver is close to essential on either track.

20. Estimated Complexity and Semester Fit Enterprise. 2,200–3,000 hours. This does not fit the student model as a whole system. Allocate as four module projects across three semesters, and scope the solver itself under Pattern C as a research prototype with an honest evaluation of its limits, not as a production timetabling system. Modules: M1+M3+M4 masters · M2+M5 workload and preferences · M7+M8 solver prototype and conflict explanation · M9+M10+M11 override, publication and views.

21. Suggested Student Team 4–5 students per module team. The solver team needs an optimisation capable student and a faculty mentor with the same background – without that mentor, do not allocate this module at all.

22. Acceptance Criteria

- Conflict-free timetable generated for one real department, minimum 200 sections, within 30 minutes
- Independent validator confirms zero hard-constraint violations
- Infeasible models produce a human-readable explanation of which constraints conflict, not a silent failure
- Manual override pins correctly and re-solve preserves pinned decisions
- Incremental re-solve after a single change completes within 2 minutes
- Workload variance across a department reported and reduced against the prior manual term
- Preference satisfaction rate reported per faculty
- New constraint types addable through configuration and documented

AI/ML potential: Learning soft-constraint weights from historical override behaviour – the overrides tell you what the institution actually values, as opposed to what it says it values.

Commercialisation: Very high value, high difficulty. Timetabling is the single most commonly outsourced problem in Indian higher education, and existing products are widely disliked. A genuinely good one is a company, not a module.

P14 – Course Registration and Elective Allocation Platform

1. Project Title High-Concurrency Course Registration and Fair Elective Allocation System. *Track J:* Next.js + NestJS + MongoDB + Redis. *Track P:* Django + PostgreSQL + Redis.

2. Project Overview Description. Handles the registration window: publishes offerings with real-time seat availability, validates eligibility against curriculum rules, and allocates over-subscribed electives by a published fair mechanism rather than first-come-first-served, with waitlists, add/drop and full auditability. *Problem.* Registration opens and tens of thousands of students hit the system in the same ten minutes. First-come-first-served rewards fast connections rather than academic merit or need, over-subscribed electives are resolved by informal appeal, and the system typically falls over – which is itself the most visible IT failure a university experiences each year. *Target users.* Students, faculty advisors, programme coordinators, registrar's office, administrators. *Expected outcome.* A registration window that does not fall over, elective allocation that is explainable and defensible, and an end to the appeals queue.

3. Project Objectives

- Publish course offerings with accurate, real-time seat availability
- Validate eligibility against curriculum and prerequisite rules, consuming P02's engine where available
- Handle extreme concurrency without seat over-allocation
- Allocate over-subscribed electives by a published, fair, auditable mechanism
- Manage waitlists with automatic promotion
- Support an add/drop window with defined rules
- Provide demand analytics that feed section planning in P13

4. Key Features Offering catalogue with live seats · eligibility pre-check before the window opens, so students know in advance · registration cart with atomic commit · preference-based batch allocation for over subscribed electives · waitlist with automatic promotion · add/drop workflow · advisor hold and approval where required · registration status and history · demand analytics and unmet-demand reporting · waiting-room mechanism under load.

5. Functional Modules

M1 Offering Catalogue Courses, sections, timings, faculty, seats, categories

Yes – build first

M2 Eligibility Pre-Check Per-student eligibility with room Depends on M1 Cart, conflict checks, atomic reasons, computed before the window
Depends on M1 seat claim Depends on M2, M3

M3 Registration Window Management

M4 Cart & Atomic Registration

Windows by cohort, staggered opening, waiting

M5 Elective Preference Collection Ranked elective preferences Depends on M1
during a preference window

priority-weighted lottery

M6 Fair Allocation Engine Batch allocation by Depends on M5
published mechanism: deferred acceptance or

M7 Waitlist Management Ordered waitlists, automatic promotion, notification Depends on M4 M8

Add/Drop Post-registration changes within policy Depends on M4

M9 Analytics & Demand Reporting Depends on M4, M6

Fill rates, unmet demand, preference satisfaction

6. User Roles & Permissions Student – own eligibility, preferences, registration, waitlist, add/drop. Faculty Advisor – advisee registrations, holds, approvals. Programme Coordinator – offerings, seats, categories, windows. Registrar's Office – full oversight, manual intervention with justification. Administrator – allocation mechanism configuration, window scheduling, system controls.

7. Technology Stack

TRACK J · Node.js + MongoDB. Mandated additions for concurrency: Redis with atomic operations or Lua scripts for seat counters – never a naive read-modify-write loop; BullMQ for the batch allocation; a virtual waiting room at the edge; horizontal scaling with a stateless API; MongoDB transactions for the commit step. Load testing with k6 at 5,000 concurrent virtual users is a named deliverable. An event-loop server absorbs many idle connections cheaply, which is why this project suits Track J.

TRACK P · Python + Django + PostgreSQL. PostgreSQL answers the correctness question in one statement: `UPDATE section SET taken = taken + 1 WHERE id = %s AND taken < capacity` – if it updates zero rows the section is full. That is atomic, needs no lock you manage, and cannot over-allocate. Make the commit idempotent with a UniqueConstraint on (student, section) so a double-submitted form cannot take two seats. Load testing with Locust. The concurrency work here is operational, not algorithmic: Unicorn worker sizing, PgBouncer connection pooling, short views, and everything slow pushed to Celery. Measure early; this is the whole point of the project.

8. Technical Requirements (deltas) *Frontend (both):* offering search with real-time seat counts – server-sent events or polling with backoff, not a per-second poll from 20,000 clients; cart with clash detection; clear waiting room experience with a position indicator; preference ranking interface. *Backend (both):* seat allocation must be atomic and over-allocation impossible under any concurrency – this is the single acceptance criterion that matters; idempotent registration commit; staggered window opening by cohort to flatten the peak; graceful degradation, so that if analytics or notifications fail, registration continues. *Allocation mechanism (both):* the fair-allocation mechanism must be published to students before the window and implemented exactly as published. Recommended: student-proposing deferred acceptance over ranked preferences with priority tiers – year, CGPA band, programme requirement – with a seeded random tiebreak that is stored and reproducible. *Data model (both):* Offering, Section, SeatCounter, RegistrationWindow, EligibilitySnapshot, Cart, Registration, ElectivePreference, AllocationRun, AllocationResult, Waitlist, AddDropRequest, AuditEntry. *API (both):* /offerings, /eligibility, /cart, /register (idempotent), /preferences, /allocations/run, /waitlist, /add-drop. *Performance (both):* 5,000 concurrent users; 500 registration commits per second; p95 under 800 ms at peak; zero over-allocation. These numbers are the project.

9. Admin Dashboard Window scheduling with cohort staggering · seat and section management with live counters · allocation mechanism configuration and dry-run · allocation run history with seed and reproducibility · live registration monitor showing throughput, errors and queue depth · manual intervention log · emergency controls – pause the window, extend it, roll back a run.

10. Reporting & Analytics Fill rate by course and section · unmet demand by elective, which directly feeds P13's section planning for the next term · preference satisfaction distribution · waitlist conversion · registration throughput and error rates during the window · add/drop patterns · eligibility failure reasons in aggregate.

11. Notifications & Communication Window opening reminder, eligibility issues before the window, registration confirmed, allocation result, waitlist promotion, add/drop outcome. Notifications must be queued and must never block the registration path.

12. Third-Party Integrations P02 eligibility engine, ERP/SIS for enrolment write-back, P13 for section and timetable data, email and SMS, SSO.

13. Expected Deliverables Per CES, plus a published allocation mechanism document written for students, a load-test report at target concurrency, and a documented runbook for the registration window itself – who watches what, and what to do when.

14. Development Milestones M0 discovery, mechanism design and publication (3w) · M1 offerings and eligibility (3w) · M2 window, cart, atomic registration (4w) · M3 load testing and hardening at target concurrency (3w) · M4 preferences and fair allocation (4w) · M5 waitlist and add/drop (3w) · M6 analytics, live-window rehearsal, production (3w).

15. Testing Per CES, plus:

- Concurrency test: 5,000 virtual users registering simultaneously; assert zero over-allocation and zero double registration.

- Chaos test: kill an application instance mid-window; assert no seat loss or corruption.
- Allocation fairness test: the mechanism produces stable, reproducible results matching a reference implementation on fixture data.
- Rehearsal: a full dress rehearsal with a real cohort before the live window is a named acceptance activity.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES, plus autoscaling for the registration window, a documented scale-up procedure, and a tested rollback.

TRACK P · Python + Django + PostgreSQL. Per CES, plus PgBouncer in front of PostgreSQL, deliberate Gunicorn worker and thread counts, and a documented scale-up procedure. Django's synchronous model makes worker sizing a first-class deployment decision here, not an afterthought.

17. Documentation Per CES, plus the mechanism specification and the window-day operations runbook.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · NestJS · Redis atomic operations and distributed locking · MongoDB transactions · high-concurrency system design · load testing with k6 · deferred acceptance · horizontal scaling.

TRACK P · Python + Django + PostgreSQL. Python · PostgreSQL conditional updates and constraints · connection pooling and worker sizing · load testing with Locust · Celery · deferred acceptance · Django deployment beyond runserver.

19. Prerequisites and Readiness Returning students with a prior project in this programme; at least one member comfortable with concurrency and load testing. Operating systems and computer networks coursework completed. *Both tracks:* concurrency, load testing and deployment tuning appear in no syllabus on this list.

20. Estimated Complexity and Semester Fit Enterprise. 1,900–2,600 hours. Do not allocate as a production system to a student team. Scope under Pattern C: a working registration system plus a rigorous load-test study at 5,000 concurrent users with a documented seat-claim correctness proof. That study is a legitimate and valuable deliverable, and it tells the institution whether a production build is worth funding. Modules across three semesters: M1+M2 offerings and eligibility · M3+M4 window and atomic registration with the load study · M5+M6 preferences and fair allocation · M7+M8+M9 waitlist, add/drop and analytics.

21. Suggested Student Team 5 students per module team – tech lead · 2 backend, one owning concurrency · frontend · QA and performance testing.

22. Acceptance Criteria

- 5,000 concurrent users sustained with p95 under 800 ms and zero errors on the registration path
- Zero over-allocation under all concurrency and chaos tests – non-negotiable
- Registration commit idempotent under duplicate submission
- Elective allocation reproducible from a stored seed and matching the published mechanism
- Waitlist promotion automatic and correctly ordered
- Live dress rehearsal completed with a real cohort without incident
- Unmet-demand report produced and usable for next-term section planning
- Emergency pause and rollback demonstrated

AI/ML potential: Demand forecasting for section planning; predicting which students are at risk of not completing registration.

Commercialisation: High value, but the concurrency requirement makes it expensive to build well. Best positioned as a module of a larger academic suite rather than a standalone sale.

P15 – Internship and AEDP Apprenticeship Management System

1. Project Title Internship, Apprenticeship and Credit-Mapped Work Experience Management Platform. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. Project Overview *Description.* Manages the full internship and apprenticeship lifecycle: an employer and opportunity registry, student applications and matching, the tripartite agreement required under UGC's Apprenticeship Embedded Degree Programme framework, hour logging, supervisor evaluation, faculty assessment, and automatic mapping of logged hours to academic credits under the National Credit Framework. *Problem.* Internships are tracked on spreadsheets and email. Nobody can answer, per student, how many hours were completed, who supervised, what was assessed, and what credit was earned. UGC's AEDP framework makes structured tripartite records and hour-to-credit mapping a compliance requirement, not a nicety – and no existing institutional system produces them. *Target users.* Students, faculty supervisors, industry supervisors, internship and placement coordinators, HoDs, deans, employers. *Expected outcome.* Every work-based learning engagement documented, hour-verified, assessed and credit-mapped, with the records an accreditation visit will ask for.

Why this ranks second in the portfolio: it is directly tied to a regulatory framework with a stated implementation date, it is genuinely absent from most institutions, and every institution that adopts AEDP will need exactly this. That combination is what makes a product.

3. Project Objectives

- Maintain an employer and opportunity registry with verification
- Manage the application, shortlist and offer workflow
- Generate and track the tripartite institution–employer–student agreement
- Log and verify working hours with supervisor confirmation
- Capture industry supervisor and faculty evaluations against a rubric
- Map verified hours to credits per the configured framework, 30 hours to 1 credit, with faculty sign-off • Support the compliance and accreditation reporting the framework requires
- Handle stipends, insurance, safety declarations and incident reporting

4. Key Features Employer registry with verification and rating · opportunity posting with skill and eligibility tags · application and offer workflow · tripartite agreement generation with e-signature · hour logging by the student and verification by the supervisor · milestone and task tracking · industry supervisor evaluation rubric · faculty assessment and moderation · automatic credit computation with a sign-off gate · attendance and leave during placement · stipend record · insurance and safety compliance checklist · incident reporting channel · completion certificate generation · compliance dashboards.

5. Functional Modules

M1 Employer &	Opportunity Registry Employers, verification,	opportunities, eligibility criteria Yes – build first
M2 Application & Selection Apply, shortlist, interview scheduling, offer, acceptance	Depends on M1 M3	
Tripartite Agreement Template generation, terms, e-signature, versioning	Depends on M2	
M4 Hour Logging & Verification	Depends on M3	
M5 Task & Milestone Tracking		
Student log, supervisor verification, disputes, caps	M6 Evaluation Industry supervisor rubric, faculty assessment, moderation	
Depends on M3 Work plan, deliverables, progress	Depends on M4, M5	
M7 Credit Mapping & Sign-Off	Hours to credits per framework; faculty approval gate; write-back	Depends on M4, M6
	declaration, incident reporting, escalation	
M8 Compliance & Safety Insurance, safety	Depends on M3	

M9 Certificates & Records Completion certificates, verifiable records,

transcripts Depends on M7

M10 Dashboards & Reporting and employer views; accreditation Depends on all
Student, department, institution reports

6. User Roles & Permissions Student – own profile, applications, agreement, hour log, tasks, evaluations, certificate. Industry Supervisor – assigned students only; verify hours, submit evaluations, report incidents; an external user with restricted, time-boxed access. Faculty Supervisor – assigned students; assess, moderate, approve credit. Internship Coordinator – employers, opportunities, agreements, compliance monitoring, escalations. Head of Department – department participation, credit and compliance reporting. Dean and Academic Office – institution-wide compliance and accreditation reports. Employer Admin – own organisation's opportunities, supervisors and students. Administrator – credit framework configuration, rubrics, agreement templates.

7. Technology Stack

TRACK J · Node.js + MongoDB. E-signature through DocuSign, Zoho Sign or a self-hosted equivalent with an audit trail; PDF generation for agreements and certificates; and a separate external-user authentication domain for industry supervisors and employers – they are not university users and must not sit inside the internal SSO trust boundary.

TRACK P · Python + Django + PostgreSQL. E-signature as above. WeasyPrint for agreements and certificates. External users get a distinct user type and permission group that cannot reach any internal view, enforced by a permission class plus middleware, with a test proving an external user hitting an internal URL receives a 403. This is exactly the users-groups-permissions material INT253 teaches, applied at its full strength.

8. Technical Requirements (*deltas*) *Frontend (both)*: mobile-first hour logging, because students log from the workplace on a phone; supervisor verification in two taps; employer portal as a distinct, simplified experience; coordinator compliance board. *Backend (both)*: hour logging with configurable caps and anomaly flags – more than 12 hours in a day, or hours logged during timetabled classes, cross-checked against P13; credit computation as a pure, auditable function with framework versioning; agreement template versioning with the executed version immutably retained. *Data model (both)*: Employer, EmployerVerification, Opportunity, Application, Offer, Agreement, AgreementTemplate, Signature, HourLog, HourVerification, WorkPlan, Task, Evaluation, Rubric, CreditMapping, CreditApproval, InsuranceRecord, SafetyDeclaration, Incident, Certificate, AuditEn try. *API (both)*: /employers, /opportunities, /applications, /agreements, /hours, /hours/:id/verify, /evaluations, /credits/compute, /credits/approve, /certificates, /reports/compliance. *Security (both)*: external supervisors get scoped, time-boxed, revocable access to assigned students only; incident reports route to a named safeguarding contact and are access-restricted; agreements and signatures are immutable once executed.

TRACK P · Python + Django + PostgreSQL. Credit computation is a function in services.py that takes hours and a framework version and returns credits, with no database writes inside it – which is what lets you test it exhaustively across every configuration. Executed-agreement immutability enforced in save().

9. Admin Dashboard Employer verification queue · agreement template management with version control · credit framework configuration – hours per credit, caps per semester, eligible engagement types · rubric builder · compliance monitor showing missing agreements, unverified hours, overdue evaluations and expired insurance · incident board · external user access management.

10. Reporting & Analytics Participation rate by programme and year · hours logged and verified · credits awarded by type and programme · employer participation and repeat rate · evaluation score distribution · conversion from internship to job offer, the number the placement cell wants · compliance gaps · accreditation-ready reports mapping students to hours, credits, supervisors and assessments.

11. Notifications & Communication New matching opportunity · application status · agreement ready for

signature · hours awaiting verification, to the supervisor · evaluation due · credit approved · insurance expiring · incident escalation.

12. **Third-Party Integrations** ERP/SIS for enrolment and credit write-back, P13 for timetable conflict checking, P06 for skill profile enrichment, e-signature provider, email and SMS, SSO for internal users, separate authentication for external users.

13. **Expected Deliverables** Per CES, plus configured agreement templates approved by the institution's legal office, a configured credit framework, and evaluation rubrics for at least three engagement types.

14. **Development Milestones** M0 discovery, AEDP framework mapping, legal review of agreement templates (3w) · M1 registry and applications (3w) · M2 tripartite agreements and e-signature (3w) · M3 hour logging and task tracking (3w) · M4 evaluations and credit mapping (4w) · M5 compliance, incidents, certificates (3w) · M6 reporting, UAT, production (3w).

15. **Testing** Per CES. Specific: credit computation correct across all framework configurations and engagement types; an external supervisor cannot access an unassigned student; hour anomalies flagged, including hours logged during scheduled classes; executed agreements immutable; incident routing reaches the safeguarding contact.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. The employer portal may be a separate deployment with its own session configuration.

TRACK P · Python + Django + PostgreSQL. Per CES. One deployment is fine; what must be separate is the authentication backend, the session cookie scope and the permission group. Document all three.

17. **Documentation** Per CES, plus a written mapping document showing how each system feature satisfies a specific UGC AEDP or NCrf requirement – this is what an accreditation team will ask for and what makes the product sellable.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Node.js · MongoDB · workflow and approval-chain design · e-signature integration · external portal security · PDF generation · compliance reporting.

TRACK P · Python + Django + PostgreSQL. Python · Django users, groups and permissions at full strength · multi-party workflow design · e-signature integration · WeasyPrint · Celery reminders · compliance reporting.

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with multi-party approval workflows and document generation; the external-supervisor portal needs someone who takes access control seriously. *Track P*: the core of this project is INT253 material; the gaps are e-signature APIs, WeasyPrint and Celery.

20. **Estimated Complexity and Semester Fit** Large. 1,200–1,600 hours. Two semesters for a team of 5. Semester 1: M1–M5 registry, applications, agreements, hour logging and tasks. Semester 2: M6–M10 evaluations, credit mapping, compliance and reporting.

21. **Suggested Student Team** 5 students – tech lead · 2 backend · frontend, internal plus external portal · QA and documentation.

22. Acceptance Criteria

- Full lifecycle completed for a real student: opportunity, application, offer, tripartite agreement e-signed, hours logged and verified, evaluations submitted, credits computed and approved, certificate issued
- Credit computation matches the configured framework exactly across all engagement types
- Credits cannot be awarded without faculty sign-off – proven by a negative test
- External supervisor access scoped to assigned students only and revocable
- Hour anomalies flagged, including conflicts with the timetable

- Compliance dashboard identifies every missing agreement, unverified hour block and overdue evaluation
- Accreditation report generated for a full academic year

AI/ML potential: Opportunity-to-student matching on skills and history; predicting completion risk from hour-logging patterns; automatic extraction of learning outcomes from supervisor evaluations.

Commercialisation: The highest-ranked project in this portfolio. Every Indian institution adopting AEDP needs this, none of them have it, and compliance-driven demand is the easiest kind to sell into.

P16 – Capstone and Sponsored Project Management Platform

1. **Project Title** Final-Year Capstone and Industry-Sponsored Project Management Platform. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. **Project Overview** *Description.* Manages final-year and capstone projects end to end: a problem-statement marketplace where faculty and external sponsors publish problems, team formation and allocation, milestone and deliverable tracking, supervisor logs, plagiarism and originality checks, evaluation by rubric with multiple examiners, IP and sponsorship agreement handling, and a public showcase of completed work. *Problem.* A university runs thousands of final-year projects a year. Allocation is chaotic, supervision is invisible until the end, evaluation is inconsistent across examiners, deliverables are collected by email, and completed work is discarded rather than showcased or commercialised. *Target users.* Students, faculty supervisors, examiners, project coordinators, HoDs, external sponsors. *Expected outcome.* Every project tracked from problem statement to evaluated deliverable, consistent evaluation, and a searchable archive of institutional project output.

3. Project Objectives

- Operate a problem-statement marketplace open to faculty and external sponsors
- Manage team formation with constraints – size, skill mix, cross-department
- Allocate teams to problems and supervisors fairly and transparently
- Track milestones, deliverables and supervision meetings
- Support originality checking and code-plagiarism detection
- Provide rubric-based multi-examiner evaluation with moderation
- Manage sponsorship agreements, IP terms and payment milestones
- Publish an archive and showcase of completed projects

4. **Key Features** Problem-statement submission and curation · sponsor onboarding with agreement and IP schedule · team formation with skill matching · preference-based team-to-project allocation · supervisor assignment and load balancing · milestone plan with deliverables · meeting log · document and code repository linkage · originality and code-similarity checking · rubric-based evaluation with multiple examiners and moderation · grading firewall – sponsors have no role in grading · viva scheduling · project archive with search · showcase site.

5. Functional Modules

M1 Problem Statement Marketplace	Submission by faculty and sponsors, curation, publication, tagging	Yes – build first
M2 Sponsor Management	Sponsor registration, agreements, IP schedule, payment milestones	Yes
M3 Team Formation	Team creation, invitations, skill mix constraints, size rules	Yes
M4 Allocation	Team preferences, supervisor capacity, fair assignment	Depends on M1, M3
M5 Milestone & Deliverable Tracking	Plan, submissions, versions, deadlines	Depends on M4
M6 Supervision Log	Meetings, notes, attendance, supervisor feedback	Depends on M4
M7 Originality & Code Similarity	Text plagiarism and code-similarity checks with Rubrics, multi-examiner scoring, moderation, viva scheduling	Depends on M5
M8 Evaluation & Moderation	showcase with permission control	Depends on M8
M9 Archive & Showcase	Searchable archive, public	Depends on M8
M10 Dashboards & Reporting	Progress, supervisor load, sponsor reporting, quality	analytics Depends on all

6. User Roles & Permissions Student – own team, project, deliverables, meeting log, feedback. Faculty School of

Computer Science & Engineering Page 57 of 75

EduRev Project List · Dual Stack · P01–P20

Supervisor – assigned teams, meeting logs, formative feedback, milestone approval. Examiner – assigned projects for evaluation only, rubric scoring. Project Coordinator – problem curation, allocation, scheduling, moderation. Head of Department – department progress, supervisor load, quality analytics. External Sponsor – own sponsored projects: progress and deliverables only, explicitly no access to grades, rubrics or evaluation. Administrator – rubrics, milestone templates, allocation configuration.

7. Technology Stack

TRACK J · Node.js + MongoDB. GitHub or GitLab API integration to link the team repository and read commit activity as a supervision signal; a plagiarism and similarity provider – Turnitin where licensed, MOSS or JPlag for code; object storage for deliverables; PDF generation for evaluation reports. The showcase is a public Next.js site with static generation.

TRACK P · Python + Django + PostgreSQL. GitHub integration with PyGithub inside Celery tasks, so a slow API call never blocks a page. Code similarity by invoking JPlag or MOSS from a task. WeasyPrint for evaluation reports. The showcase is the one part that argues for Path P3 – a Next.js static site consuming the archive API – though Django with aggressive caching is entirely adequate.

8. Technical Requirements (*deltas*) *Frontend (both)*: problem marketplace with faceted search; team formation with invitations; milestone board per team; examiner scoring interface designed for speed, because an examiner evaluates 20 projects in a day; showcase as a public, statically served site. *Backend (both)*: allocation respecting team preferences, supervisor capacity and expertise; deliverable versioning with immutable submission timestamps; similarity checks queued; evaluation aggregation with configurable moderation rules – for example, flag when two examiners differ by more than 15%. *Data model (both)*: ProblemStatement, Sponsor, SponsorshipAgreement, Team, TeamMember, ProjectAllocation, Supervisor, Milestone, Deliverable, DeliverableVersion, MeetingLog, SimilarityReport, Rubric, Evaluation, Moderation, VivaSchedule, ArchiveEntry, ShowcasePermission. *API (both)*: /problems, /sponsors, /teams, /allocations/run, /milestones, /deliverables, /similarity/check, /evaluations, /archive, /showcase. *Security (both)*: the grading firewall is an access control requirement, not a policy statement – sponsors must be structurally unable to reach evaluation data, covered by an explicit negative test.

TRACK P · Python + Django + PostgreSQL. Implement the firewall as a sponsor-specific permission class *and* a filtered `get_queryset` that never joins evaluation tables – two independent layers. Deliverable timestamps set server-side with `auto_now_add` and made read-only in every serializer and form.

9. Admin Dashboard Problem curation queue · sponsor and agreement management with payment milestone tracking · allocation run and manual adjustment · supervisor load balancing · milestone template management · rubric builder · moderation queue for divergent scores · similarity review · showcase publication approval.

10. Reporting & Analytics Project progress by department and milestone · supervisor load and meeting frequency · evaluation score distribution and examiner variance, which identifies inconsistent examiners – genuinely useful, politically sensitive, and access-restricted · similarity finding rates · sponsor participation and repeat rate · sponsorship revenue and payment status · project outcome tracking – publication, patent, product, startup.

11. Notifications & Communication Problem published matching interests · team invitation · allocation result · milestone due and overdue · deliverable submitted · similarity flag · evaluation assigned · viva scheduled · sponsor progress digest.

12. Third-Party Integrations GitHub or GitLab, plagiarism provider, ERP/SIS for enrolment and grade write-back, e-signature for sponsorship agreements, calendar for viva scheduling, SSO plus separate external authentication for sponsors.

13. Expected Deliverables Per CES, plus configured rubrics and milestone templates for at least two programmes, and a working public showcase.

14. Development Milestones M0 discovery and rubric or milestone design (2w) · M1 marketplace, sponsors, teams (3w) · M2 allocation and milestones (3w) · M3 supervision log and similarity checking (2w) · M4 evaluation and moderation (3w) · M5 archive, showcase, analytics, UAT, production (3w).

15. Testing Per CES. Specific: a sponsor cannot access any evaluation data under any path, tested exhaustively; allocation respects supervisor capacity; deliverable submission timestamps immutable; moderation triggers on divergent scores; the showcase respects per-project publication permission.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. The showcase deployed as a statically generated site, rebuilt on publication.

TRACK P · Python + Django + PostgreSQL. Per CES. If the showcase stays in Django, cache it aggressively and invalidate on publication; if it moves to Next.js under Path P3, it becomes a second deployment – decide deliberately, not by drift.

17. Documentation Per CES, plus a written statement of the grading firewall – what a sponsor can and cannot see – that the project coordinator can verify without reading code.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Next.js including static generation · Node.js · MongoDB · Git provider APIs · file and version management · rubric and evaluation modelling · allocation algorithms.

TRACK P · Python + Django + PostgreSQL. Python · Django permissions and querysets · PyGithub · Celery · similarity tooling · allocation algorithms · INT257 if the showcase is built in Next.js.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with Git provider APIs; the grading firewall needs someone who will test access control properly. *Track P gaps:* GitHub API, similarity tools, Celery.

20. Estimated Complexity and Semester Fit Medium, upper end. 700–900 hours. One semester for a team of 5 with the showcase (M9) as a stretch goal, or two semesters for a team of 4. A strong early allocation – the sponsor is the department itself, so requirements are easy to obtain.

21. Suggested Student Team 5 students – tech lead · 2 backend · frontend and showcase · QA and documentation.

22. Acceptance Criteria

- Problem statements published by both faculty and an external sponsor
- Teams formed and allocated respecting size, skill-mix and supervisor capacity constraints
- Sponsor demonstrably cannot access grades or rubrics – proven by negative tests across every endpoint
- Deliverable versions immutable with reliable submission timestamps
- Similarity check runs and produces a reviewable report
- Multi-examiner evaluation aggregated with moderation triggered on divergence
- Public showcase live with per-project permission control respected
- Sponsorship agreement and payment milestones tracked

AI/ML potential: Problem-to-team matching on skills; automatic project abstract and tag generation; early detection of at-risk projects from commit and meeting patterns.

Commercialisation: Strong. Capstone management is universal, and the sponsored-project and IP-agreement modules are unusual – most competing tools handle only the academic side.

P17 – Student Earnings, Freelancing and Enterprise Tracking Platform (EdRev)

1. **Project Title** Student Earning, Freelancing and Venture Tracking Platform with Opportunity Bid Desk. *Track J:* Next.js + Node.js + MongoDB. *Track P:* Django + PostgreSQL.

2. **Project Overview** *Description.* The operating system for an institutional student-earning programme: tracks each student's progression through a training and tier ladder, records verified platform earnings, runs an opportunity Bid Desk that sources and allocates external gigs to matched students with SLA tracking, monitors academic safeguards – hour caps, CGPA gates, exam blackouts – and reports programme outcomes. *Problem.* Institutions running student freelancing and revenue-generation initiatives operate them on spreadsheets and WhatsApp groups. Nobody knows who is earning, how much, from where, whether hour caps are respected, or whether participants' grades are holding. Without that data the programme cannot be defended, improved or scaled. *Target users.* Students, cell coordinators, faculty mentors, guild leads, placement office, deans. *Expected outcome.* A measurable earning programme: verified earnings, an operating opportunity pipeline, enforced academic safeguards, and outcome data linking participation to placement results.

Design constraint that shapes the data model: the institution takes no share of individual student platform earnings. Earnings are recorded for measurement, safeguarding and outcome reporting only – never for deduction, and there must be no billing or deduction pathway anywhere in the system. Institutional revenue-share applies only to work routed through an institutional agency account, which is a separate, explicitly flagged engagement type.

3. Project Objectives

- Manage cohort intake, training progression and a tiered capability ladder
- Operate an opportunity Bid Desk: source, triage, allocate with SLA, and track outcome •
- Record and verify student earnings across multiple platforms and dimensions
- Enforce academic safeguards: hour caps, CGPA gates, exam blackout periods
- Track quality signals – platform ratings, disputes, delivery outcomes
- Manage a proposal budget and report cost-per-win
- Report programme outcomes including the placement differential

4. **Key Features** Cohort and application management · training module tracking with completion gates · tier ladder with evidence-based promotion and demotion · platform account registry, declared and never credential holding · earnings log with document evidence and verification · Bid Desk: opportunity board, triage, one-bid-per-institution allocation rule, SLA timers, win and loss tracking · proposal library and editor review queue · proposal budget management · hour logging with caps · exam blackout enforcement · CGPA integration and gate · dispute and rescue-bench tracking · compliance checklist · mentor assignment · programme analytics.

5. Functional Modules

M1 Cohort & Student Registry
M2 Training & Tier Progression

M3 Platform Account Registry
M4 Earnings Log & Verification

M5 Bid Desk – Opportunity Board Applications, selection, cohorts, equity-seat tracking, undertakings

Modules, completion, artefacts, tier criteria, promotion and demotion with evidence	evidence upload; coordinator verification; currency handling Sourcing intake, triage,	Depends on M3
Declared accounts per platform and dimension; never stores credentials	client-quality scoring, board states Yes – build first Depends on M1	Yes – fully independent, the highest-value module
Self-reported earnings with	Depends on M1	

M6 Allocation & SLA Match by niche, capacity, tier and development need; one-bid rule; SLA timers; allocation log visible to the	cohort Depends on M2, M5	
	Templates, case-study bank, editor review queue with turnaround tracking	Depends on M6
M7 Proposal Library & Review		
M8 Budget Management Proposal budget, spend per student, cost-per-win	Depends on M6	
M9 Academic Safeguards Hour caps, exam blackout, CGPA gate, automatic suspension	rescue-bench assignment, remediation fund Depends on M1	Depends on M4
M10 Quality & Disputes Ratings, disputes,		
M11 Compliance Tracker PAN, bank, payment rail, tax position per student	Depends on M1	
M12 Analytics & Outcome Reporting	Funnel, earnings distribution, equity metrics, placement	differential Depends on all

6. **User Roles & Permissions** Student – own profile, tier, earnings, allocations, proposals, hours, compliance checklist. Bid Desk Analyst – opportunity board, triage, allocation within SLA. Proposal Editor – review queue only. Guild Lead or Mentor – assigned students’ progression and quality. Cell Coordinator – full programme operations, verification, budget, disputes. Faculty Director – safeguards, academic health, escalations. Placement Office – outcome analytics, read-only, aggregate. Dean – programme reporting, aggregate only. Administrator – configuration.

7. Technology Stack

TRACK J · Node.js + MongoDB. BullMQ for SLA timers and opportunity-feed polling, RSS and email parsing for opportunity ingestion, currency conversion with a rate-history table, and object storage for earnings evidence.

TRACK P · Python + Django + PostgreSQL. Celery Beat for SLA sweeps and feed polling; feedparser for RSS ingestion. All money in DecimalField – never FloatField. Earnings are stored in the original currency with the historical rate on the row and the local value computed from it, never today’s rate. The exam blackout is derived automatically from the academic calendar, not entered by hand.

On both tracks the system must never store or request platform login credentials. Say this explicitly in the posting, because a team will otherwise propose scraping accounts.

8. **Technical Requirements (deltas)** *Frontend (both):* Bid Desk board optimised for speed, because an analyst triages 150 opportunities a week; mobile earnings entry with evidence photo; student dashboard showing tier progress and next criteria; transparent allocation log visible to all cohort members. *Backend (both):* SLA timers persisted and restart-safe; one-bid-per-opportunity enforced at the data level – a unique constraint, not application logic alone; earnings stored in original currency with a stored rate and computed local value; hour caps enforced with a hard block and an override requiring the Faculty Director. *Data model (both):* Cohort, Application, Student, Undertaking, TrainingModule, ModuleCompletion, TierAssessment, PlatformAccount,

EarningsEntry, EarningsVerification, FxRate, Opportunity, OpportunityTriage, Allocation, SlaEvent, Proposal, ProposalReview, BudgetLedger, HourLog, BlackoutPeriod, AcademicSnapshot, Rating, Dispute, RescueAssignment, Compliance. *API (both)*: /cohorts, /students/:id/tier, /earnings, /earnings/:id/verify, /opportunities, /opportunities/:id/allocate, /proposals, /budget, /hours, /analytics/programme. *Privacy (both)*: individual earnings are sensitive. Aggregate-only access for the placement office and deans; no institution-wide earnings leaderboard anywhere in the product – a deliberate product decision, not an oversight, stated in the specification so nobody helpfully adds one.

TRACK P · Python + Django + PostgreSQL. The one-bid rule is a UniqueConstraint on the opportunity: the database refuses the second bid even if two analysts click in the same instant. Prove it with a TransactionTestCase using real concurrent processes.

9. Admin Dashboard Cohort and intake management · tier criteria configuration · training module management

School of Computer Science & Engineering Page 61 of 75

EduRev Project List · Dual Stack · P01–P20

· opportunity source configuration and feed health · SLA configuration · budget allocation and spend · verification queue · dispute and rescue board · safeguard breach monitor · compliance completion tracker.

10. Reporting & Analytics Intake-to-first-earning funnel with stage conversion · time to first payment and to first contract · earnings distribution by median and decile, never a leaderboard · win rate and cost-per-win by tier · dimension mix per student · SLA compliance · equity metrics: share of earnings reaching reserved seat students · academic health of participants against matched non-participants · placement differential · programme cost per student reaching a defined earning threshold.

11. Notifications & Communication Opportunity allocated with SLA countdown · proposal review complete · earnings verification required · tier promotion eligible · hour cap approaching · exam blackout starting · CGPA gate triggered · compliance item outstanding · dispute raised.

12. Third-Party Integrations ERP/SIS for CGPA and enrolment – essential for the safeguard module, academic calendar for blackout periods, email and RSS ingestion for opportunity sourcing, an exchange-rate API, email and SMS, SSO. No platform account integrations – declared accounts only.

13. Expected Deliverables Per CES, plus configured tier criteria, training modules and an opportunity source list for a live cohort.

14. Development Milestones M0 discovery and programme model (2w) · M1 cohorts and tier progression (3w) · M2 Bid Desk, allocation, proposals (4w) · M3 accounts and earnings with verification (3w) · M4 safeguards and compliance (2w) · M5 budget, quality, disputes (2w) · M6 analytics, UAT, production (3w).

15. Testing Per CES. Specific: the one-bid rule cannot be violated even under concurrent allocation; SLA timers survive restart; the hour cap blocks and requires a Director override; exam blackout auto-applies from the academic calendar; currency conversion uses the stored historical rate; no endpoint returns another student's earnings.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. Feed pollers and SLA workers separate from the API.

TRACK P · Python + Django + PostgreSQL. Per CES. Celery worker and a single Beat instance; the feed poller is a periodic task, not a long-running process.

17. Documentation Per CES, plus a written statement of the no-deduction and no-leaderboard decisions, so a future team does not reverse them by accident.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Node.js · MongoDB · BullMQ · workflow and SLA systems · multi-currency financial data modelling · dashboard analytics · privacy-conscious API design.

TRACK P · Python + Django + PostgreSQL. Python · Django · database constraints as correctness tools · Celery Beat ·

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with background job queues and multi-currency financial data; the Bid Desk needs someone who will design for speed of use rather than completeness of features.

20. **Estimated Complexity and Semester Fit** Large, lower end. 850–1,100 hours. Two semesters for a team of 4, or one semester for a team of 5 with quality, disputes and budget (M8+M10) deferred. The Bid Desk (M5+M6+M7) is independently buildable and is the highest-value module.

21. **Suggested Student Team** 5 students – tech lead · 2 backend · frontend · QA and documentation.

22. Acceptance Criteria

- Full student journey: application, selection, training, tier promotion, allocation, proposal, earning recorded and verified

- One-bid-per-opportunity enforced at the database level and proven under concurrency
- SLA timers accurate and restart-safe
- Hour caps hard-blocked; override requires the Faculty Director role and is audited
- Exam blackout applied automatically from the academic calendar
- Earnings stored in original currency with stored rate; local-currency reporting reconciles exactly
- No leaderboard exists anywhere and no earnings-deduction pathway exists – verified by review
- Placement differential report produced comparing participants with a matched cohort

AI/ML potential: Opportunity-to-student matching on niche and history; win-probability scoring on opportunities to prioritise Bid Desk effort; early identification of students at risk of dropping out before first earning.

Commercialisation: Moderate to strong. Very few institutions run structured earning programmes today, but the number is growing and none of them has a system. Early-mover advantage in a small but real category.

P18 – CO-PO Attainment and Accreditation Management System

1. Project Title Outcome-Based Education, CO-PO Attainment and Accreditation Evidence Platform. *Track J:* Next.js + NestJS + MongoDB. *Track P:* Django + PostgreSQL.

2. Project Overview *Description.* Implements outcome-based education end to end: defines Programme Outcomes, Programme Specific Outcomes and Course Outcomes; maps CO to PO with articulation strength; maps every assessment question to a CO and Bloom's level; computes direct attainment from actual marks and indirect attainment from surveys; produces attainment reports, gap analysis and continuous-improvement action tracking; and assembles the evidence packs NBA and NAAC actually ask for. *Problem.* CO-PO attainment is computed once a year in enormous, error-prone spreadsheets by whichever faculty member is least able to refuse. The calculation is opaque, the evidence is assembled in a panic before an accreditation visit, and the

continuous-improvement loop that outcome-based education is supposed to create does not exist. *Target users.* Faculty, course coordinators, programme coordinators, HoDs, IQAC and accreditation cell, deans. *Expected outcome.* Attainment computed continuously and automatically from real assessment data, gaps visible during the semester rather than after it, and accreditation evidence assembled on demand.

Why this ranks first in the portfolio: it is mandatory – NBA and NAAC require it – universally painful, currently solved with spreadsheets everywhere, and identical in shape at every institution in India. That is the definition of a sellable product.

3. Project Objectives

- Maintain PO, PSO and CO definitions with programme and curriculum versioning
- Maintain CO-PO articulation matrices with strength levels and justification
- Map every assessment component and question to a CO and a Bloom's taxonomy level • Compute direct attainment from marks against configurable thresholds and rubrics • Capture indirect attainment through course-exit, programme-exit, alumni and employer surveys • Combine direct and indirect attainment with configurable weights
- Identify gaps against targets and track continuous-improvement actions to closure
- Generate accreditation evidence packs in the required formats

4. Key Features PO, PSO and CO authoring with versioning · CO-PO articulation matrix editor with justification capture · assessment blueprint mapping question to CO to Bloom level to marks · marks ingestion from existing examination systems · direct attainment computation with configurable attainment levels · survey builder and distribution for indirect attainment · combined attainment with weights · target setting and gap analysis · continuous-improvement action tracking with closure evidence · attainment trend across semesters and cohorts · accreditation evidence pack generation · faculty course-file assembly.

5. Functional Modules

M1 Outcome Definition PO, PSO and CO with versioning and approval workflow	M4 Marks Ingestion Import from the examination system, or manual and bulk entry with validation Yes – build first Depends on M1 Depends on M1
M2 Articulation Matrix CO-PO mapping with strength 1, 2 or 3 and justification	Depends on M3
M3 Assessment Blueprint Components, questions, CO mapping, Bloom level, marks distribution	
M5 Direct Attainment Engine	Question to CO to PO attainment with configurable thresholds Depends on M2, M3, M4
M6 Survey Engine Builder, distribution, response collection for indirect attainment	Yes
M7 Indirect Attainment Survey-derived attainment mapped to CO and PO	Depends on M6, M2 School of Computer Science & Engineering Page 64 of 75

Science & Engineering Page 64 of 75

EduRev Project List · Dual Stack · P01–P20

M8 Combined Attainment & Targets	comparison, gap identification	files, trend analysis
M9 Continuous Improvement	Action plans against gaps, ownership, closure evidence, verification	Depends on M5, M7 Depends on M8 Depends on all
M10 Evidence Pack & Reporting	Accreditation-format reports, course	
Weighted combination, target		

6. User Roles & Permissions Faculty – own courses: CO definition, blueprint, marks, attainment view, action plans. Course Coordinator – all sections of a course, moderation across sections. Programme Coordinator – PO

and PSO, articulation matrices, programme attainment. HoD – department attainment, gaps, action closure. IQAC or Accreditation Cell – institution-wide attainment, evidence packs, survey administration. Dean – institutional reporting. Administrator – attainment levels, thresholds, weights, formats. Student – respond to surveys; optionally view own CO attainment.

7. Technology Stack

TRACK J · Node.js + MongoDB. NestJS for domain complexity. Excel import and export as a first-class capability (ExcelJS) – every institution's existing data is in spreadsheets and the migration path is the adoption path. PDF and DOCX report generation in accreditation formats. A spreadsheet-like grid component (AG Grid or Handsontable) for marks and blueprint entry, because faculty will reject anything slower than Excel.

TRACK P · Python + Django + PostgreSQL. Python is a real advantage here. Marks import and export with openpyxl, and the attainment arithmetic with pandas inside a Celery task. Reports with WeasyPrint for PDF and python-docx for Word. Marks entry still needs a real JavaScript grid – Handsontable or similar – which is the one part of this project outside Django.

8. Technical Requirements (deltas) *Frontend (both):* articulation matrix as an editable grid; blueprint builder with running marks totals and CO coverage validation; marks entry that is genuinely faster than Excel, with keyboard navigation and paste support – this determines whether faculty adopt the system at all; attainment dashboards with drill-down from PO to CO to question. *Backend (both):* attainment computation as a pure, versioned, auditable function – every reported number must be traceable to the under lying question-level marks; configurable attainment level definitions, for example Level 3 meaning 60% of students scoring above 60%; recomputation on marks correction with a full history; batch computation across a department as a background job. *Data model (both):* Programme, ProgrammeOutcome, ProgrammeSpecificOutcome, Course, CourseOutcome, ArticulationMatrix, AssessmentComponent, Question, QuestionCoMapping, BloomLevel, StudentMark, AttainmentRun, DirectAttainment, Survey, SurveyResponse, IndirectAttainment, CombinedAttainment, Target, Gap, ImprovementAction, EvidenceArtefact, AuditEntry. *API (both):* /outcomes/*, /articulation/*, /blueprints/*, /marks/import, /attainment/compute, /attainment/runid, /surveys/*, /improvement-actions, /evidence-packs/generate. *Auditability (both):* this is the defining non-functional requirement. An accreditation team will ask how a number was produced, and the system must answer down to the individual question and student mark, for any attainment figure, for any past semester.

TRACK P · Python + Django + PostgreSQL. Store every computation as an AttainmentRun carrying the input version, so a figure from three years ago can still be explained. The computation function takes marks and a configuration and returns numbers, with no database writes inside it.

9. Admin Dashboard Attainment level and threshold configuration · direct and indirect weight configuration · outcome approval workflow · survey administration · evidence pack template management · computation run history with version and reproducibility · data completeness monitor showing which courses are missing blueprints or marks · action closure tracker.

10. Reporting & Analytics CO attainment per course and section · PO and PSO attainment per programme and cohort · attainment trend across semesters · gap analysis against targets · Bloom-level distribution across assessments, which exposes the common finding that most questions test recall · section-to-section variance for the same course · action plan closure rate · complete accreditation evidence packs in NBA and NAAC formats · faculty course files assembled automatically.

11. Notifications & Communication Blueprint due, marks entry due, attainment computed, gap identified, action plan assigned, action overdue, survey opening and closing, accreditation deadline countdown.

12. Third-Party Integrations Examination and ERP system for marks – essential, since manual entry is a fallback and not the design; LMS for assessment data; survey distribution by email; SSO. Excel import and export as the universal fallback for every integration.

13. Expected Deliverables Per CES, plus a fully worked attainment computation for one real programme for one completed semester, reconciled against the institution's existing spreadsheet calculation. That reconciliation is the acceptance artefact – if the numbers do not match, either the spreadsheet or the system is wrong, and finding out which is the point.

14. **Development Milestones** M0 discovery, outcome-based education workshop, format collection (3w) · M1 outcomes and articulation (3w) · M2 blueprints and marks ingestion (4w) · M3 direct attainment engine (3w) · M4 surveys, indirect and combined attainment (4w) · M5 continuous improvement (2w) · M6 evidence packs, reconciliation, UAT, production (4w).

15. **Testing** Per CES. The reconciliation test is the primary acceptance gate: system-computed attainment must match the institution's manually computed figures for a full semester, or every difference must be explained and attributed. Also: recomputation after a marks correction produces a correct new run with history preserved; attainment traceable to question level for a randomly selected figure.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. Batch attainment computation on separate workers.

TRACK P · Python + Django + PostgreSQL. Per CES. Celery worker for batch computation and report generation; WeasyPrint needs its system libraries in the image, which is the one non-obvious line in the Dockerfile.

17. **Documentation** Per CES, plus a written computation methodology document – the single most requested artefact in an accreditation visit.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · NestJS · MongoDB aggregation · complex calculation engines with auditability · Excel import and export · report generation · high-performance data grids · an understanding of outcome-based education is a genuine advantage and should be asked about.

TRACK P · Python + Django + PostgreSQL. Python · pandas and openpyxl · Django ORM aggregation · auditable calculation design · WeasyPrint and python-docx · a JavaScript data grid · an understanding of outcome-based education is a genuine advantage and should be asked about.

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member must work closely with an IQAC-experienced faculty member throughout – the domain is unintuitive and getting the attainment arithmetic wrong invalidates the whole system. Comfort with spreadsheet-grade data grids and report generation required on either track.

20. **Estimated Complexity and Semester Fit** Large. 1,300–1,700 hours. Two to three semesters for a team of 5. Semester 1: M1+M2+M3 outcomes, articulation and blueprints. Semester 2: M4+M5 marks ingestion and the direct attainment engine, with the reconciliation test. Semester 3: M6–M10 surveys, combined attainment, improvement tracking and evidence packs.

21. **Suggested Student Team** 5 students – tech lead · 2 backend, one owning the attainment computation · frontend with high-performance grids · QA and documentation. Plus a named IQAC faculty domain adviser, who is not optional.

22. Acceptance Criteria

- Attainment computed for one real programme for one semester, reconciling with the institution's existing manual calculation or explaining every difference
- Any attainment figure traceable to question-level marks in under three clicks

- Attainment levels, thresholds and direct or indirect weights configurable without deployment
- Marks entry demonstrably faster than the existing spreadsheet workflow, in a timed usability test with real faculty
- Excel import handles the institution's actual existing files
- NBA and NAAC format evidence packs generated and accepted as usable by the IQAC
- Improvement actions tracked to closure with evidence
- Recomputation after a marks correction preserves the prior run

AI/ML potential: Automatic CO mapping suggestion for questions from question text; Bloom-level classification of questions; identifying which assessment patterns predict poor attainment.

Commercialisation: The highest in the portfolio. Mandatory compliance, universal pain, spreadsheet incumbency, identical requirements across thousands of institutions. Build it multi-tenant from the first commit.

P19 – AI-Assisted Question Bank and Assessment Platform

1. **Project Title** Institutional Question Bank, Blueprint-Driven Paper Generation and Online Assessment Platform. *Track J:* Next.js + NestJS + MongoDB. *Track P:* Django + PostgreSQL + Redis.

2. **Project Overview** *Description.* A governed question bank tagged by course outcome, Bloom level, difficulty and topic; blueprint-driven automatic paper generation with balance constraints; a moderation and approval workflow; online assessment delivery with configurable integrity controls; auto-evaluation for objective and code questions; and item analysis that feeds question quality back into the bank. *Problem.* Question papers are

written from scratch each term, stored on personal drives, unbalanced across outcomes and Bloom levels, and never analysed for item quality. There is no institutional memory of what has been asked, and no defence when a paper is challenged. *Target users*. Faculty, course coordinators, examination cell, students, HoDs. *Expected outcome*. A growing institutional question asset, papers generated in minutes against a validated blueprint, and item statistics that improve question quality each cycle.

3. Project Objectives

- Maintain a governed question bank with rich tagging and version control
- Enforce a moderation and approval workflow before a question is examinable
- Generate papers automatically against a blueprint with CO, Bloom, difficulty and marks constraints • Deliver online assessments securely with configurable integrity controls
- Auto-evaluate objective and programming questions; support rubric-based manual evaluation • Produce item analysis – difficulty index, discrimination index, distractor analysis
- Feed question-to-CO performance data into P18

4. Key Features Question authoring with rich media, mathematical notation and code · tagging by CO, Bloom, difficulty, topic and marks · version control and usage history, which prevents the same question appearing in consecutive terms · moderation workflow · blueprint definition and validation · automatic paper generation with constraint satisfaction · paper variants and shuffling · secure online delivery with timer and autosave · configurable integrity controls · auto-evaluation for multiple-choice and code · rubric-based manual evaluation with moderation · item analysis · question quality scoring · bulk import.

5. Functional Modules

M1 Question Bank & Tagging Authoring, media, tags, versioning, usage history Yes – build first	M9 Item Analysis Difficulty index, discrimination index, distractor analysis, quality score M10 Reporting & Integration Result publication, CO-wise performance, export to P18 Depends on M1
M2 Moderation Workflow Review, approval, rejection with feedback, approval status gating	Yes
M3 Blueprint Definition CO, Bloom, difficulty and marks distribution rules with validation	Depends on M1, M2, M3
M4 Paper Generation Constraint-satisfying selection, variants, shuffling, preview	Depends on M4
M5 Assessment Delivery Scheduling, secure delivery, timer, autosave, resumption	Depends on M5
M6 Integrity Controls Tab-switch detection, randomisation, time limits, IP restriction, optional monitored mode	Depends on M5
M7 Auto-Evaluation Objective scoring; code execution against test cases in a sandbox	Depends on M5
M8 Manual Evaluation Rubric scoring, anonymised marking, moderation, re-evaluation	Depends on M7, M8 Depends on M9

6. User Roles & Permissions Faculty – author questions, propose papers, evaluate assigned scripts. Course Coordinator – moderate questions, approve blueprints and papers. Examination Cell – scheduling, delivery, result publication, integrity review. Student – take assessments, view results and feedback within release rules. HoD – quality analytics. Administrator – configuration, sandbox management, integrity policy.

7. Technology Stack

TRACK J · Node.js + MongoDB. A sandboxed code-execution service with isolated containers and strict resource, time and network limits – the security-critical component, specified explicitly. Mathematical notation rendering, a rich-text editor with media, object storage for question media, and Redis for assessment session state. An event-loop server

holds thousands of concurrent exam sessions cheaply, which is why this project suits Track J at its stated scale.

TRACK P · Python + Django + PostgreSQL. Paper generation is a constraint problem – use OR-Tools or PuLP, both Python, in a Celery task. Session state in Redis with periodic persistence to PostgreSQL, so a Redis restart does not destroy an exam in progress. The sandbox is driven by the docker Python SDK. At 3,000 concurrent students, Django's synchronous model is the binding constraint – either accept ASGI with Channels for the exam interface, or size Gunicorn workers carefully and prove the target in the load test before committing.

8. Technical Requirements (deltas) Frontend (both): question authoring with mathematical notation, code blocks and image upload; blueprint builder showing live coverage; an assessment interface robust to connection loss – local autosave with resumption, because a student losing 40 minutes of work to a dropped connection is the failure that kills adoption; evaluation interface optimised for volume. **Backend (both):** paper generation as constraint satisfaction over the tagged bank, with an explanation when constraints cannot be met; code execution sandboxed with no network access, hard CPU, memory and time limits, a read-only filesystem, a non-root user, and container-per-submission isolation; assessment sessions resumable; result computation with re-evaluation history. **Data model (both):** Question, QuestionVersion, QuestionTag, UsageHistory, ModerationReview, Blueprint, Paper, PaperVariant, Assessment, AssessmentSession, Response, Evaluation, Rubric, IntegrityEvent, ItemStatistic, Result. **API (both):** /questions, /questions/:id/moderate, /blueprints, /papers/generate, /assessments, /assessments/:id/session, /responses, /evaluate, /item-analysis. **Security (both):** the code sandbox is the highest-risk surface – the week-2 design review must record the isolation model explicitly. Question content is confidential before an examination: encrypted at rest, access-controlled, released on a schedule, with every pre-release access logged. **Performance (both):** 3,000 concurrent students in an assessment; the code evaluation queue clearing within 5 minutes of submission close.

9. Admin Dashboard Question bank health – coverage by CO, Bloom and difficulty, identifying gaps · moderation queue · blueprint library · paper generation history · assessment scheduling calendar · live assessment monitor · integrity event review · item quality report · sandbox resource monitor.

10. Reporting & Analytics Question bank coverage and gaps · Bloom distribution across a semester's assessments, which feeds P18 and usually reveals over-reliance on recall · item difficulty and discrimination with flagging of poor items · distractor effectiveness · CO-wise student performance · assessment result distribution · integrity event patterns · faculty contribution to the bank.

11. Notifications & Communication Question moderation required, paper approved, assessment scheduled and starting, submission received, results published, re-evaluation outcome, item flagged as poor quality.

12. Third-Party Integrations P18 for CO-wise performance export – a primary integration; ERP for enrolment and result write-back; LMS; SSO; the code-execution sandbox service.

13. Expected Deliverables Per CES, plus a seeded question bank of at least 500 moderated questions across two courses and a working sandbox for at least three programming languages.

14. Development Milestones M0 discovery and taxonomy (2w) · M1 bank and moderation (4w) · M2 blueprints and paper generation (3w) · M3 delivery and integrity (4w) · M4 sandbox and auto-evaluation (3w) · M5 manual evaluation and item analysis (3w) · M6 reporting, load test, UAT, production (3w).

15. Testing Per CES. Specific: sandbox escape testing is a named security deliverable – attempt network access, filesystem access, fork bombs and resource exhaustion; a 3,000-concurrent-student load test; session resumption after simulated connection loss; paper generation satisfies all blueprint constraints or explains why not; item statistics verified against a manual calculation.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. Sandbox executors on isolated hosts, never on the application host.

TRACK P · Python + Django + PostgreSQL. Per CES. Sandbox workers on a dedicated Celery queue running on separate machines, never on the web host. If the exam interface uses Channels, document the ASGI split as in P07.

17. Documentation Per CES, plus documented sandbox security architecture.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · NestJS · MongoDB · container sandboxing and secure code execution · constraint satisfaction · real-time assessment delivery at scale · psychometrics basics · rich-text and formula editors.

TRACK P · Python + Django + PostgreSQL. Python · container and Linux security, the docker SDK · OR-Tools or PuLP · Redis and session state · Celery queues · Channels or careful worker sizing · psychometrics basics.

19. Prerequisites and Readiness Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member with container and Linux security depth for the sandbox, and one comfortable with real-time session handling at scale. Operating systems coursework completed. *Track P:* this project has the longest gap list on that track – container security, Redis, OR-Tools and Channels are all outside the four courses.

20. Estimated Complexity and Semester Fit Large, upper end. 1,400–1,800 hours. Allocate as three module projects. The question bank and blueprint modules (M1–M4) are a clean one-semester project for 5 students. Assessment delivery (M5+M6+M8) is a second. The code-execution sandbox (M7) must be scoped under Pattern C as a prototype with a documented security evaluation – a student-built sandbox should not run untrusted code in production without a professional security review.

21. Suggested Student Team 5 students per module team. The sandbox team needs a security-capable mentor; without one, allocate the module as a study rather than a build.

22. Acceptance Criteria

- 500 questions authored, moderated and approved across two courses
- Paper generated satisfying all blueprint constraints, with a clear explanation when constraints are unsatisfiable
- 3,000 concurrent students sustained without session loss
- Assessment session resumes correctly after a simulated connection loss
- Code execution fully sandboxed – sandbox escape testing passed with evidence
- Item analysis matches manual calculation on a verified fixture
- CO-wise performance exported to P18 in the agreed format
- Question content inaccessible before scheduled release; all pre-release access logged

AI/ML potential: Substantial – question generation from course material with mandatory faculty review, automatic CO and Bloom tagging, difficulty prediction before first use, semantic duplicate detection in the bank, short-answer auto-evaluation with human moderation. Every AI-generated question must pass human moderation before becoming examinable; build that as a hard gate, not a setting.

Commercialisation: Strong, but a competitive category. The differentiator is blueprint-driven generation tied to CO-PO attainment (P18), which the generic assessment platforms do not do.

P20 – Campus Resource, Lab and Facility Booking Platform

1. Project Title Unified Campus Resource, Laboratory and Facility Booking Platform. *Track J:* Next.js + Node.js + MongoDB + Redis. *Track P:* Django + PostgreSQL.

2. Project Overview Description. One booking system for every bookable campus resource – classrooms, laboratories, equipment, seminar halls, sports facilities, meeting rooms, vehicles – with availability rules, approval workflows, conflict prevention, check-in verification, utilisation analytics and integration with the timetable so that teaching allocations are respected automatically. *Problem.* Resources are booked by phone call, email, register book and WhatsApp, in departmental silos. Double-booking is routine, utilisation is unknown,

expensive equipment sits idle because nobody knows it exists, and no-shows block resources others needed. *Target users.* Students, faculty, staff, resource custodians, department heads, facility managers. *Expected outcome.* Zero double-bookings, visible utilisation, and expensive resources actually used.

3. Project Objectives

- Maintain a unified catalogue of bookable resources with attributes and images
- Define availability rules, booking windows, duration limits and blackout periods
- Prevent conflicts absolutely, including against the published timetable
- Support configurable approval workflows by resource type and requester role
- Verify actual use through check-in and release unused bookings automatically
- Report utilisation, idle capacity and no-show behaviour
- Handle recurring bookings, maintenance windows and consumables

4. Key Features Resource catalogue with search and images · availability rules and blackout periods · calendar booking with real-time conflict prevention · timetable-aware availability – a room teaching at 10 a.m. is simply not offered · approval workflows by type and role · recurring bookings · check-in and check-out with QR verification · automatic release of un-checked-in bookings · no-show tracking and progressive restriction · maintenance scheduling · consumable and accessory tracking · utilisation dashboards · departmental quotas.

5. Functional Modules

M1 Resource Catalogue Types, attributes, images, capacity, features, custodians, location	duration and lead-time limits, blackouts, quotas Yes – build first; shared with P13's M4 Depends on M1
M2 Availability & Rules Hours, booking windows,	
M3 Booking Engine Conflict-free reservation with atomic slot claim	Depends on M2
M4 Timetable Integration Consume the published timetable as a hard availability constraint	resource type and requester role Depends on M3 Depends on M3
M5 Approval Workflow Configurable chains by	
M6 Check-in & Auto-Release	automatic release, check-out tracking Depends on M3 Depends on M1
M7 Maintenance & Downtime QR check-in, grace period,	Scheduled maintenance, breakdown reporting, downtime
M8 Consumables & Accessories	Stock, issue with booking, return Yes no-shows, demand against supply Depends on M3, M6
M9 Analytics & Reporting Utilisation, idle capacity,	

6. User Roles & Permissions Student – browse and book permitted resources, check in, view own bookings. Faculty and Staff – broader booking rights, recurring bookings, book on behalf of a class. Resource Custodian

– own resources: availability, approvals, maintenance, consumables. Department Head – departmental quotas and utilisation. Facility Manager – cross-campus utilisation, policy configuration. Administrator – resource types, workflows, quota configuration.

7. Technology Stack

TRACK J · Node.js + MongoDB. A calendar and scheduling component (FullCalendar or equivalent), QR generation and scanning, Redis for atomic slot locking, and iCal export so bookings appear in personal calendars. Optional door-access or IoT sensor integration for automatic occupancy verification as a v2 feature.

TRACK P · Python + Django + PostgreSQL. PostgreSQL solves the core problem outright. Store each booking as a DateTimeRangeField and add an ExclusionConstraint on (resource, period) with the overlap operator. The database

then refuses to store two overlapping bookings for the same resource – not your code, the database. No lock to manage, no race condition to reason about, and it holds no matter how many users click at once. Learning this one constraint is worth the project on its own. QR codes with the qrcode library; auto-release with a Celery Beat sweep; iCal export with the ics library.

8. Technical Requirements (*deltas*) *Frontend (both)*: resource search with facets and photographs, because people do not book equipment they cannot picture; calendar view with drag-to-select; mobile check-in by QR scan; custodian approval queue. *Backend (both)*: atomic slot reservation – double-booking must be structurally impossible; the timetable consumed as a hard constraint with automatic refresh on repub location; an auto-release job releasing bookings not checked into within the grace period; recurring bookings expanded with per-occurrence conflict handling. *Data model (both)*: ResourceType, Resource, ResourceAttribute, Custodian, AvailabilityRule, Blackout, Quota, Booking, BookingSlot, RecurrenceRule, ApprovalWorkflow, Approval, CheckIn, NoShow, MaintenanceWindow, Consumable, Issuance, UtilisationSnapshot. *API (both)*: /resources, /resources/:id/availability, /bookings, /bookings/:id/approve, /bookings/:id/check-in, /maintenance, /analytics/utilisation. *Performance (both)*: 500 concurrent booking attempts at a peak – a lab slot release – with zero double-booking.

TRACK J · Node.js + MongoDB. Achieved with a unique index on the resource-timeslot pair plus a Redis lock; the lock is the part that needs care, because it must be released on every failure path.

TRACK P · Python + Django + PostgreSQL. Achieved with one constraint and no lock at all. The team's job is to prove it: fire 500 simultaneous attempts at one slot and assert exactly one booking exists.

9. Admin Dashboard Resource catalogue with bulk import · availability rule and quota configuration · approval workflow builder · live booking board · no-show monitor with restriction management · maintenance calendar · utilisation heat map · consumable stock alerts.

10. Reporting & Analytics Utilisation by resource, type, department and period · idle capacity ranking, which identifies expensive equipment nobody uses – usually the report that pays for the project · no-show rate by user and resource · demand against supply, showing which resources need more capacity · approval turnaround · maintenance downtime · departmental quota consumption.

11. Notifications & Communication Booking confirmed, approval required, approved or rejected, reminder before start, check-in required with grace countdown, auto-released, maintenance scheduled, resource unavailable.

12. Third-Party Integrations P13 for the published timetable – essential; ERP and SSO for identity and role; calendar export (iCal, Google, Outlook); optional door-access and IoT occupancy; email and SMS.

13. Expected Deliverables Per CES, plus a seeded catalogue covering at least three resource types with real data.

14. Development Milestones M0 discovery and resource inventory (2w) · M1 catalogue and rules (2w) · M2 booking engine and timetable integration (3w) · M3 approvals and check-in (2w) · M4 maintenance and consumables (2w) · M5 analytics, UAT, production (2w).

15. Testing Per CES. Specific: 500 concurrent booking attempts on the same slot produce exactly one booking; timetable conflicts never offered as available; auto-release fires correctly after the grace period; a recurring booking with a mid-series conflict handled correctly; quota enforcement.

16. Deployment

TRACK J · Node.js + MongoDB. Per CES. Redis is on the critical path for booking – monitor it accordingly.

TRACK P · Python + Django + PostgreSQL. Per CES. Redis is needed only for Celery, not for booking correctness, which is one less thing that can take the system down.

17. Documentation Per CES, plus a written explanation of the conflict-prevention mechanism – the artefact that shows the team understood why it works.

18. Skills Students Need

TRACK J · Node.js + MongoDB. TypeScript · Node.js · MongoDB unique indexing and transactions · Redis locking · calendar and scheduling UI · QR handling · location modelling.

TRACK P · Python + Django + PostgreSQL. Python · Django · PostgreSQL range types and exclusion constraints · Celery Beat · QR generation · CSE326 for the calendar interface.

19. **Prerequisites and Readiness** Completed Data Structures and DBMS; at least one prior project; comfortable with Git branching and pull requests. One member comfortable with atomic operations and constraints – preventing double-booking is the whole technical point; one comfortable with calendar interfaces. *Track P:* nearly everything sits inside INT253 and CSE326; the gaps are range types, Celery and QR generation.

20. **Estimated Complexity and Semester Fit** Medium. 550–700 hours. One semester for a team of 4–5. A strong semester-1 allocation: unambiguous, immediately useful, and the resource catalogue (M1) is reused by P13 later.

21. **Suggested Student Team** 4–5 students – tech lead · backend, owning booking atomicity · frontend and calendar · QA and documentation.

22. Acceptance Criteria

- 500 concurrent attempts on one slot yield exactly one booking – proven by test
- Timetable-occupied slots never offered
- Approval workflows configurable per resource type without deployment
- QR check-in works on mobile; un-checked-in bookings auto-release after the grace period
- No-show tracking and progressive restriction functioning
- Utilisation and idle-capacity reports produced across at least three resource types
- Recurring bookings created with correct per-occurrence conflict handling

Note on sequencing. M1 (Resource Catalogue) is shared with P13's M4. Build it once, in whichever project starts first, and have the other consume it. Say this in both project briefs – if two teams build two resource catalogues, you will spend the next three years reconciling them.

AI/ML potential: Demand forecasting for capacity planning; no-show prediction to enable controlled overbooking of high-demand resources.

Commercialisation: Solid. A universal campus need, straightforward to build well, and it integrates naturally into a larger suite.

Appendix – Track selection summary

Read this with §1.8 and §1.9. Path is the recommended delivery path for a team on Track P; Track J teams use Path J1 (Next.js full-stack) unless the project notes say otherwise.

P01 Academic Risk Radar Large 5 P1 Either Background jobs; aggregation and window functions
P02 Academic Advisor & Degree Planning P03 Hostel Allocation & Roommate Matching

Large 5/mod P1/P2	Either Writing a rule interpreter allocation algorithm	P04 Hostel Room Exchange	Small 3 P1 Track P
Medium 4–5 P1	Either Jobs, PDF generation,	Transactions and row locking	
	Medium 4 P1 Track P	forecasting	
P05 Mess Demand & Food-Waste	Service workers, offline sync,		
	access control		
P06 Placement Skill Matcher		Medium 4–5 P1	Either WebSockets, ASGI, field encryption, restart-safe timers
P07 Health & Emergency Response			
Large 5 P1/P2	Either API framework, full-text search,		
P08 Research Intelligence Hub	Large 4–5/mod	P1 Track P	matching, entity resolution
		External APIs, fuzzy	
P09 Lost & Found	Small 2–3 P1/P3	Either Image handling; structural field redaction	
P10 Campus Issues & Network Quality		mapping, QR	
Medium 5 + 3 P1	Either Restart-safe SLA sweepers,		
P11 Alumni Mentorship	Medium 4–5 P1/P2	Either OAuth and calendar APIs; privacy testing	
P12 CertiScholar Document	Verification		OCR, computer vision, queued pipelines
	Large 5 P2 Track P		
	Timetable	5/mod	Constraint programming with OR-Tools
P13 Faculty Workload &	Enterprise 4–	P1 Track P	
P14 Course Registration	Enterprise 5/mod P1	Track J	Concurrency, load testing, deployment tuning
P15 Internship & AEDP	Large 5 P1	Either E-signature, external-user auth, document generation	
P16 Capstone & Sponsored Projects		similarity tooling	
Medium 4–5 P1/P3	Either Git provider APIs,		
	Large 4–5 P1 Track P		multi-currency modelling
P17 Student Earnings (EdRev)	Database constraints,		
P18 CO-PO Attainment	Large 5 P1 Track P	P19 Question Bank & Assessment	
Spreadsheet handling, auditable computation, data grids		Large 5/mod P1/P2 Track J	Container security, constraint solving, scale
P20 Resource & Facility Booking	Medium 4–5 P1 Track P		constraints
	Range types and exclusion		

Track J paths. J1 = Next.js full-stack (INT252 + INT257) · J2 = React SPA + Node API (INT252, backend self-taught). Track P paths. P1 = Django templates + CSE326 · P2 = Django REST Framework + React (INT252) · P3 = Django REST Framework + Next.js (INT257), for pages needing SEO or server rendering.

“Suits” is a recommendation, not a restriction. Every project can be built on either track. Where the column names a track, that track removes a structural obstacle – a second service, a hand-built guarantee, or a missing library – rather than merely being more familiar. A team with real depth in the other track should choose the other track and record why in the design document.

